

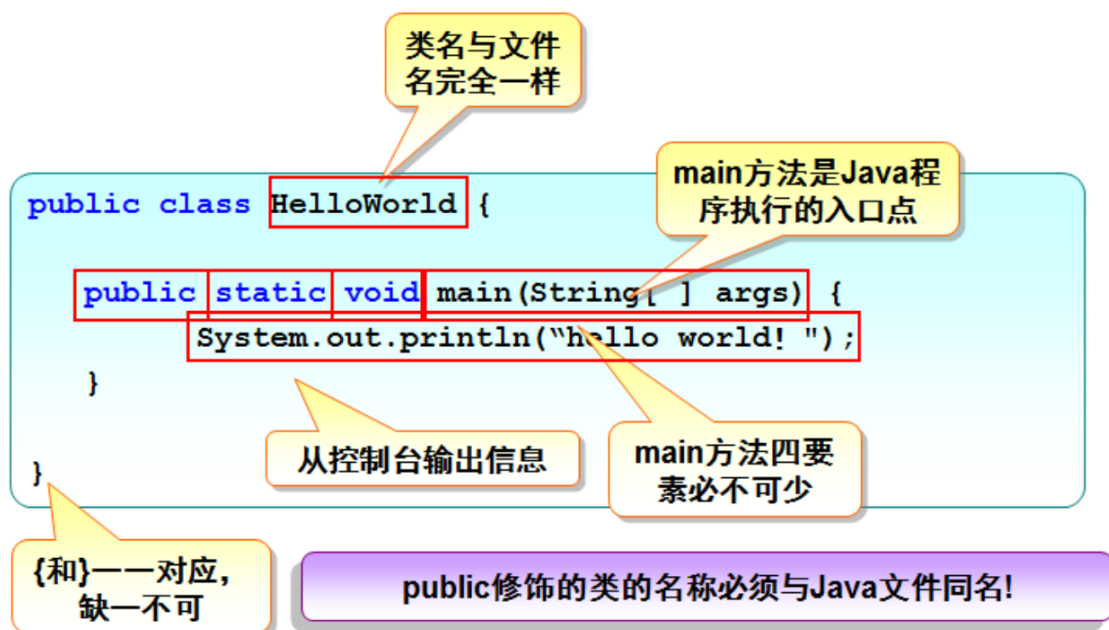
面向对象程序设计 (Java)

Lesson 1 认识Java

Hello, World!

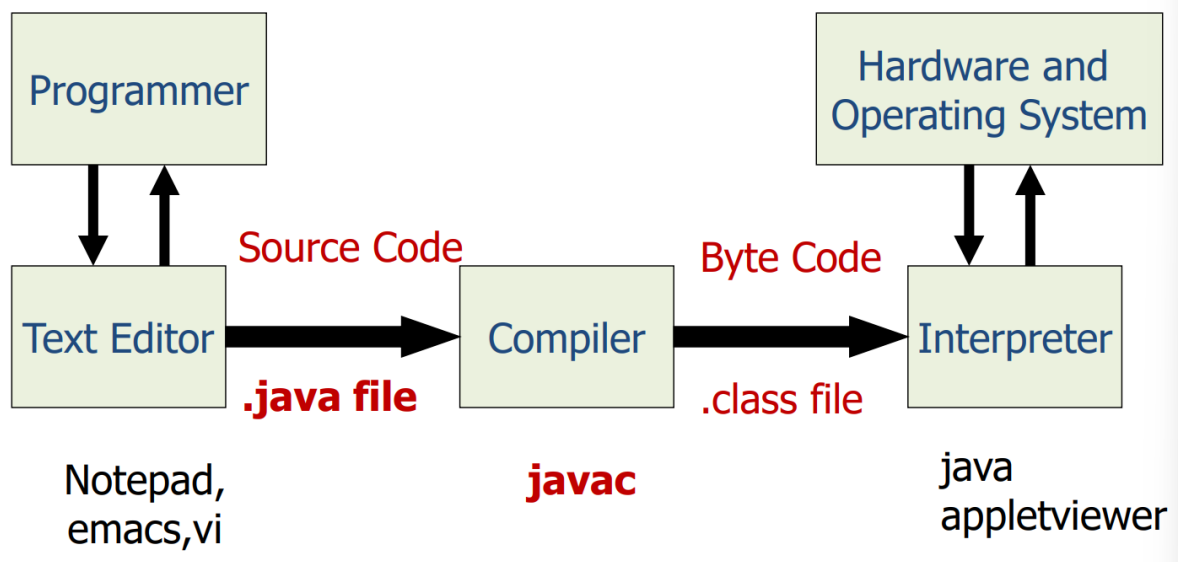
注意：虽然是一个简单的在main函数内写的Hello, World程序，但也需要放在类中，并且**类名**需要和**文件名**保持完全一致

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("hello world!");  
    }  
}
```

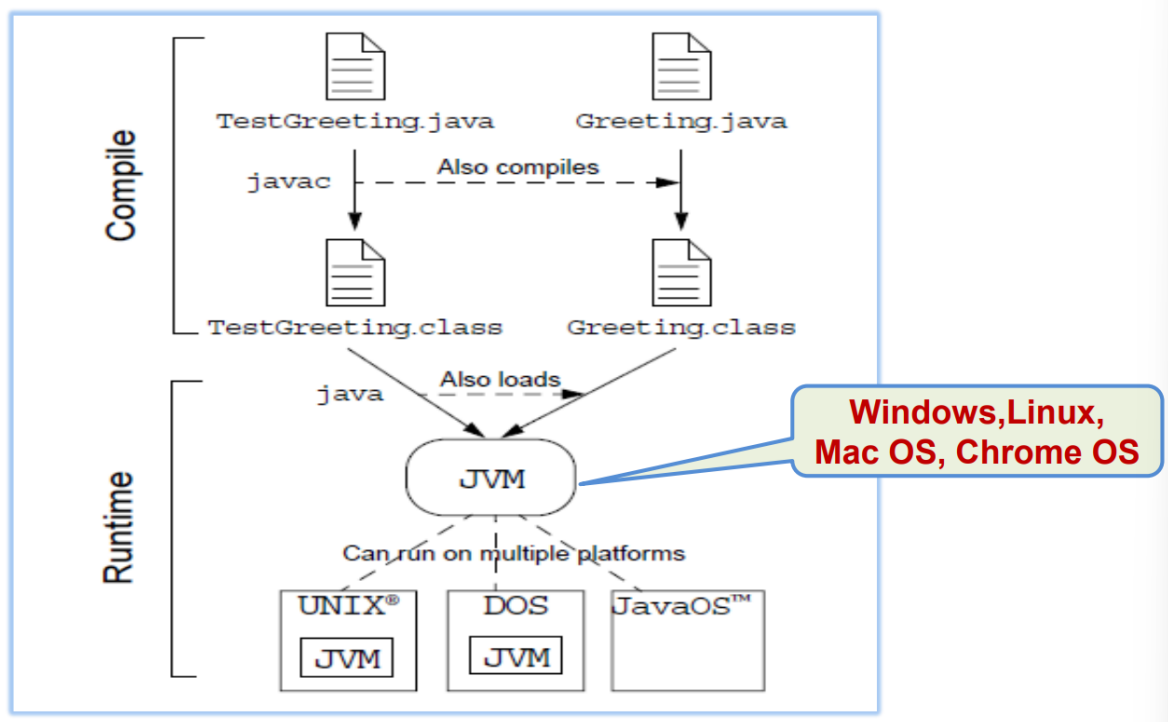


Java Compiler and Interpreter

- Java不是像C语言生成运行文件，也不是像Python直接编译执行在操作系统上，Java需要先通过javac编译器编译为.class文件，之后在JVM虚拟机上进行汇编执行



- Java是编译和解释相结合
- 无论哪种高级语言，都必须提供相应的编译器和解释器
- Java语言编写的程序使用的是编译与解释相结合的方式运行的
- JVM中提供的JIT（即时编译方式）将字节码直接转化成高性能的本地机器码，即JIT使得Java程序既能跨平台又能高速运行



Lesson 2 封装数据为类

类与对象的关系

- 类和对象之间是**抽象和具体的关系**，类是总称，对象是个体，因此**对象**也叫做**实例**

Java程序的基本结构

- 重点：**包命名常采用顶级域名作为前缀，随后紧跟其他的内容
即，需要**逆序设置**

1、包的声明；//指定类所在的位置

- 包命名常采用顶级**域名**作为前缀，例如com, net, org, edu, gov, io等，随后紧跟公司/组织/个人名称以及功能模块名称（cn.edu.buaa.oop）

2、类的导入；//用到其它位置中的类

3、类的定义；//核心部分

Java源文件基本语法

{<包声明>}
{<导入声明>}
{<类声明>}

- 示例：HelloWorld.java 文件：

```
1  package cn.edu.buaa.oop.lesson_1;
2
3  import cn.edu.buaa.oop.lesson_2.Student;
4
5  public class HelloWorld {
    Run | Debug
6      public static void main(String[] args) {
7          System.out.println("Hello world!");
8          Student stu1 = new Student();
9      }
```

【类】命名规范

1. 首字母大写
2. 一个类名包含两个以上名词，建议使用驼峰命名 (Camel-Case)法 (**S**tudent**T**est)

【类】修饰符

修饰符分为访问控制符和类型说明符

【类】访问控制符

- `public`，即公共类
- 默认，没有访问控制符

PS:

(重点：一个Java源程序只能由一个 `public` 类，且这个方法一般含有 `main` 方法

- ① 一个类被定义为公共类，就表示它能够被其它所有的类访问和引用。
- ② 在一个Java源程序中只能有一个public类，这个类一般含有main方法。
- ③ 不用public定义的类，其只能被同一个包中定义的类访问和引用。
- ④ 在一个JAVA程序中可以定义多个这样的类。

PS: 设计原则

- 一. 取有意义的名字。
- 二. 尽量将数据设计为私有属性。
- 三. 尽量对变量进行初始化。
- 四. 类的功能尽量单一（原子、细粒度）。

【成员变量】修饰符

有四种

- ① public、
- ② private、
- ③ protected、
- ④ final、
- ⑤ this(大多数可以省略，实例变量)、
- ⑥ static(类变量)、
- ⑦ 默认

【成员变量】访问级别

Modifier	Class	Package	Subclass	World
public	Y	Y	Y	Y
protected	Y	Y	Y	X
no modifier	Y	Y	X	X
private	Y	X	X	X

【成员变量】命名规范

- 首字母小写
- 一个变量名包含两个以上名词，从第二词开始建议使用驼峰命名(Camel-Case)法（studentName）

PS：设计原则

- 一. 取有意义的名字。
- 二. 尽量将数据设计为私有属性。
- 三. 尽量对变量进行初始化。

【成员方法】命名规范

- 首字母小写
- 一个方法名包含两个以上名词，从第二词开始建议使用驼峰命名(Camel-Case)法（getStudentName）
- 方法名第一个单词为动词

局部变量和成员变量的区别

内存中的位置不同

- 成员变量存储在堆内存的对象中
- 局部变量存储在栈内存的方法中

初始化不同

- 成员变量因为存储在堆内存中，所有成员变量具有**默认的初始化值**
- 局部变量没有默认的初始化值，必须**手动给其赋值**才可以使用

方法的重载

- 方法的重载是指一个类中可以定义有相同的名字，但参数不同的多个方法，调用时会根据不同的参数表选择对应的方法。
- 调用的方法实现由参数**个数**和参数**类型**确定

toString()方法

重点：创建类时没有定义 `toString()` 方法，**输出对象时输出对象的哈希码值（对象的内存地址）**，相当于调用输出函数时都是默认 `toString()` 转化为 `String` 类型进行输出

- 在java中，所有对象都有默认的`toString()`这个方法
- 创建类时没有定义`toString()`方法，**输出对象时会输出对象的哈希码值（对象的内存地址）**
- 它通常只是为了方便输出，比如
`System.out.println(xx)`，（xx是对象），括号里面的“xx”如果不是String类型的话，就自动调用xx的`toString()`方法
- 它只是sun公司（Oracle）开发java时为了方便所有类的字符串操作而特意加入的一个方法

构造函数

特点：

- **与类同名**
- **没有任何返回值**
- 语法结构与一般的方法相同

构造方法的重载

每个类至少有一个构造方法

并且：构造方法可以重载，且通常是重载的

- 不带参数构造方法（默认构造方法）
- 带参数构造方法

为了简化对象初始化的代码

不带参数的构造方法 or 默认构造方法

- 无参数构造函数创建对象时，成员变量的值被赋予了数据类型的隐含初值

— 例如：整型是 0，实型是 0.0，复合类型是 null。

变量类型	默认值	变量类型	默认值
byte	0	short	0
int	0	long	0L
float	0.0f	double	0.0d
char	'\u0000'	boolean	false
引用类型	null		

- 在Java中，如果一个类没有明显的表明哪一个类是它的父类，Object 类就是它的父类

练习：

```

public class Sample {
    private int x;
    ✓ public Sample() {
        x = 1;
    }
    ✓ public Sample(int i) {
        x = i;
    }
    ✓ public int Sample(int i) {
        x = i;
        return x++;
    }
    ✓ private Sample(int i, String s){}
    ✓ public Sample(String s,int i){}
    ✗ private Sampla(int i){
        x=i++;
    }
    ✓ private void Sampla(int i){
        x=i++;
    }
}

```

无参构造方法

带参构造方法

不是构造方法

带参构造方法

带参构造方法

名称与类名不相同

不是构造方法

Lecture 3 封装数据为类

题外话：

懒加载：Lazy Load，对某资源只在需要时才寻找其存在并初始化；对立面是预加载。

预加载：提前加载好所有资源，等待使用资源的那一刻。

对于一些使用频率较低但初始化开销很大的资源，懒加载可以避免他们给程序的初始化增加过多的负担。

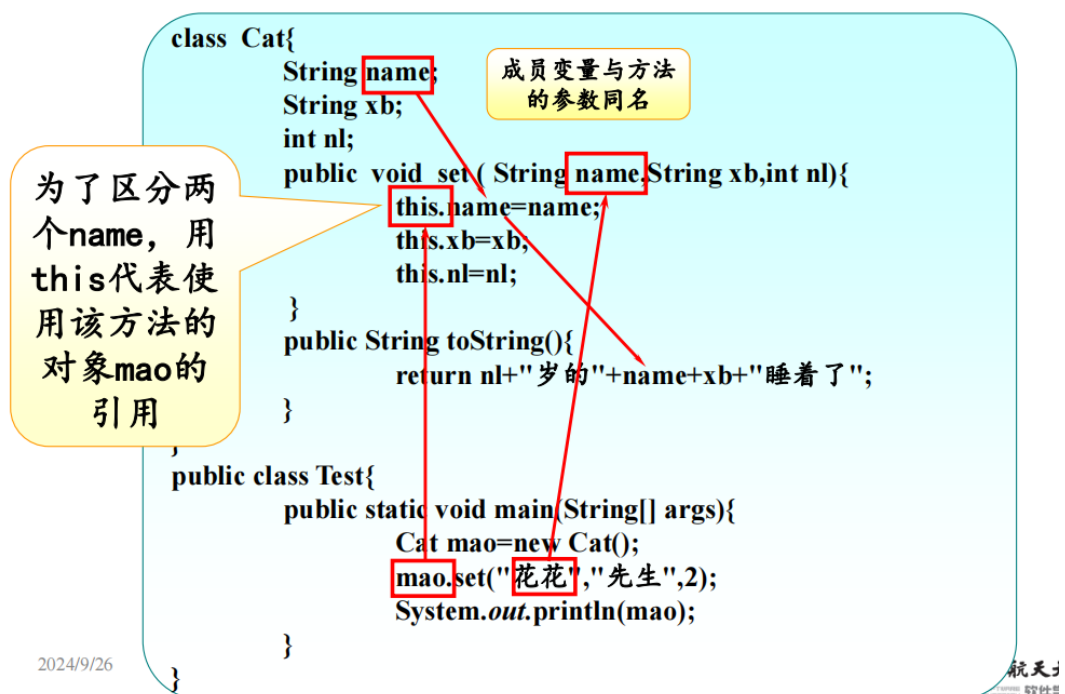
JVM 的类加载是懒加载，只有在程序第一次使用到某个类时才去尝试读取其 .class 文件。类加载只会进行一次，这一次类加载会完成所有的静态初始化工作。更多内容会在后续课程讲解 RTTI 和反射的时候提到。

50例如，在游戏编程中，当某一个类的所有实例都使用同一批贴图文件时，可以将贴图资源声明为 static 属性并直接（或在 static 块）初始化。让类的贴图属性引用这些静态资源，这样就可以避免为每一个对象构造单独的贴图文件导致的内存浪费和时间浪费。

this引用

- Java每个类都默认地具有 `null`, `this`, `super` 三个域，所以在任何类中都可以不加说明直接引用它们
 - `null`：代表“空”，用在定义一个对象但尚未为其开辟内存空间时
 - `this`：是常用的指代**本类对象**的关键字，可以看作一个变量，值即为**当前对象的引用**

作用：处理方法中**成员变量**与**局部变量**重名的问题



内存的管理

- 一种方法是由**程序员**在编写程序时**显式释放**，例如 `C/C++`
- 一种方法是由**语言的运行机制**自动完成，例如 `Java`

Java的内存管理（垃圾自动回收机制）

- **Java虚拟机 (JVM)** 后台线程负责内存的回收
- **垃圾强制回收：**Java系统提供方法 `System.gc()` 和 `Runtime.gc()` 方法来强制立即回收垃圾

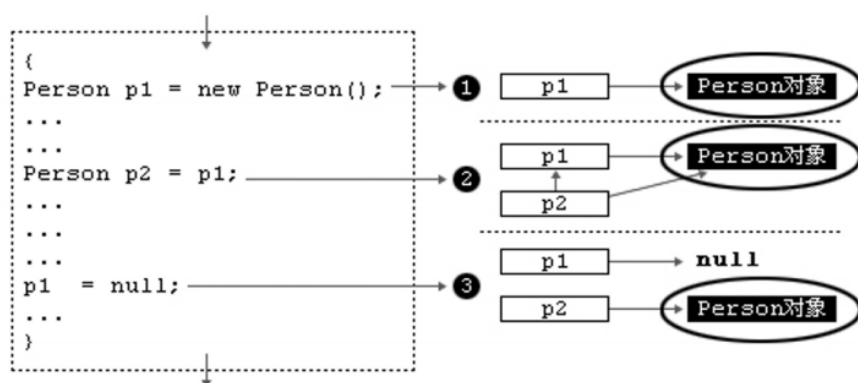
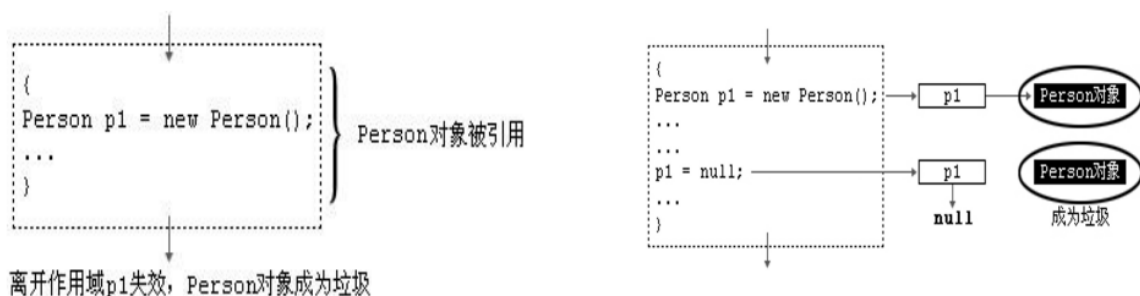
判断一个存储单元是否是垃圾的**依据**：**该存储单元所对应的对象是否仍被程序所用**

判断一个对象是否仍为程序所用的**依据**：**是否有引用指向该对象**

- Java提供的**类似析构的函数**：`protected void finalize()`

JVM在回收对象存储单元之前先调用该对象的 `finalize` 方法，如果该对象没有定义该方法，则调用该对象默认的 `finalize` 方法

example



```
3 public class Student {
4
5     @Override
6     protected void finalize(){
7         System.out.println(x: "Finalize function is running!");
8     }
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85 public static void main(String[] args){
86     Student stu = new Student(studentId: "S20371324". studentName: "Elsa".
87         studentAge: 19, idNumber: "401
88         new Address(street: "Xueyuan
89         Building", unitNo: 501));
90     // int age = stu.getAge();
91     // Student.PhoneNumber book = stu.new PhoneNumber("15554673672");
92     System.out.println(stu);
93     new Student(studentId: "S20371325", studentName: "Tom",
94         studentAge: 19, idNumber: "401
95         new Address(street: "Xue
96         Building", unitNo: 501));
97     stu = null;
98     System.gc();
99 }
```

Invoke twice

Student Tom is destroyed immediately

Student Elsa is destroyed

匿名函数

- 我们也可以不定义对象的句柄，而直接调用这个方法。

这样的对象叫做匿名对象


```
teacher1.addStudent(new Student(studentId: "S20371324",  
studentName: "Elsa", studentAge: 19, idNumber: "400401....", addr));
```

匿名对象

```
39 | public void addStudent(Student stu) {  
40 |     students.add(stu);  
41 | }
```

深拷贝 vs 浅拷贝

- **基本类型**作为参数传递时，是将变量的空间中值**复制**了一份进行传递
- 当**引用变量**作为参数传递时，为浅拷贝，其实是将引用变量空间中的**内存地址**赋值了一份传递

对象序列化

Java 提供了一种对象序列化的机制，该机制中，一个对象可以被表示为一个字节序列，该字节序列包括该对象的数据、有关对象的类型的信息和存储在对象中数据的类型。将序列化对象写入文件之后，可以从文件中读取出来，并且对它进行反序列化，也就是说，对象的类型信息、对象的数据，还有对象中的数据类型可以用来在内存中新建对象。

序列化输出至文件：

```
Run | Debug  
7 | public static void main(String [] args)  
8 | {  
9 |     Employee e = new Employee(name: "Reyan Ali", address: "Phokka Kuan,  
    Ambehta Peer", number: 103);  
10 |  
11 |     try  
12 |     {  
13 |         FileOutputStream fileOut =  
14 |             new FileOutputStream(name: "/tmp/employee.ser");  
15 |         ObjectOutputStream out = new ObjectOutputStream(fileOut);  
16 |         out.writeObject(e);  
17 |         out.close();  
18 |         fileOut.close();  
19 |         System.out.println(x: "Serialized data is saved in /tmp/employee.ser");  
20 |     } catch (IOException i)  
21 |     {  
22 |         i.printStackTrace();  
    }
```

读入序列化对象


```

6      public static void main(String[] args) {
7          Employee e = null;
8          try {
9              FileInputStream fileIn = new FileInputStream(name: "/tmp/employee.
              ser");
10             ObjectInputStream in = new ObjectInputStream(fileIn);
11             e = (Employee) in.readObject();
12             in.close();
13             fileIn.close();
14         } catch (IOException i) {
15             return;
16         } catch (ClassNotFoundException c) {
17             return;
18         }
19         System.out.println(x: "Deserialized Employee...");
20         System.out.println("Name: " + e.name);
21         System.out.println("Address: " + e.address);
22         System.out.println("Number: " + e.getNumber());

```

静态属性和静态方法

- Java中把**代表类范围信息**的变量用关键字 `static` 修饰
- 用 `static` 修饰的属性（变量）称为**静态属性**，或**类变量**
- 用 `static` 修饰的方法称为**静态方法**，或**类方法**（静态方法里无 `this`）
- **静态数据以及全局数据**存在数据区

example - 静态变量

统计类变量有多少个

```

3      public class People{
4          String name;
5          String sex;
6          int age;
7          public static int count;
8          public int _count;
9
10         public People(String name, String sex, int age){
11             this.name = name;
12             this.age = age;
13             this.sex = sex;
14             count ++;
15             _count ++;
16         }
17
18         public String toString(){
19             String ret = "name: " + name + " sex: " + sex + " age: " + age;
20             return ret;
21         }
22     }

```

- **静态方法与非静态方法的区别**:
 - 静态方法可以直接由**类名**调用，而非静态方法只能在实例化对象调用
 - 静态方法不可以访问普通成员变量或非静态方法

example - 静态方法

```
3 public class Test {
4     public int sum(int a, int b){
5         return a+b;
6     }
7
8     Run | Debug
9     public static void main(String []args){
10         int res = sum(a: 1, b: 2);
11         System.out.println("the sum is " + res);
12     }
13 }
```

Compilation Error: Cannot make a static reference to the non-static method sum(int, int) from the type Test

单例模式

- 问题：构造方法可以使用private修饰吗？

- 问题：希望程序使用者不能随意自己创建对象，

只能获取一个单独的对象，即一个类在内存中只有一个实例（对象）存在，如何实现？

例如：比如多程序访问同一个配置文件，希望多程序操作的都是同一个配置文件中的数据，那么就需要保证该配置文件对象的唯一性。

又比如系统日志，多线程访问一个程序，需要记录日志，需要把日志单例化。你不希望看到N多个日志文件实例吧？

- 一个类的内存中只有一个实例对象存在，该类**一般没有属性**
- **无法继承**，所以**无法扩展**，无法更改它的实现

实现方式：

- “饿汉式”单实例模式

```

public class Singleton
{
    //静态的。保留自身的引用。 类加载时就初始化
    private static Singleton test = new Singleton();
    //必须是私有的构造函数
    private Singleton(){}
    //公共的静态的方法。
    public static Singleton getInstance() {
        return test;
    }
}

```

- 是否 Lazy 初始化：否
 - 是否多线程安全：是
 - 描述：这种方式比较常用，但容易产生垃圾对象。
 - 优点：没有加锁，执行效率会提高。
 - 缺点：类加载时就初始化，浪费内存。
 - 特点：它基于 classloader 机制避免了多线程的同步问题，不过，instance 在类装载时就实例化
- “懒汉式”单实例模式

<pre> public class Singleton { //静态的。保留自身的引用。 private static Singleton test = null; //必须是私有的构造函数 private Singleton(){} //公共的静态的方法。 public static Singleton getInstance() { if(test == null) { test = new Singleton(); } return test; } } </pre>	懒汉式：单例的延迟加载方式。 延迟：需要的时候，再创建
---	--

- 是否 Lazy 初始化：是
- 是否多线程安全：否
- 描述：这种方式是最基本的实现方式，这种实现最大的问题就是不支持多线程。因为没有加锁 synchronized，所以严格意义上它并不算单例模式。
- 特点这种方式 lazy loading 很明显，不要求线程安全，在多线程不能正常工作。

对于单例的延迟加载方式会出现多线程中的安全问题，需要进行如下修改
(同时仅允许单线程进入)

```
class Single2
{
    private static Single s = null;
    private Single(){}
    public static Single getInstance()
    {
        if(s==null)
        {
            synchronized(Single.class)
            {
                if(s==null)
                {
                    s = new Single();
                }
            }
        }
        return s;
    }
}
```

应用场景

- 只需要使用一个单独的资源，并且需要共享这个单独资源的状态信息时，就能用到单例模式。
- 需要频繁的进行创建和销毁的对象；
- 创建对象时耗时过多或耗费资源过多，但又经常用到的对象；
- 工具类对象；
- 频繁访问数据库或文件的对象。

代码块

构造代码块会在创建对象时被调用，每次创建时都会被调用，优先于类构造函数执行

```
3 public class CodeBlocks {
4     private int number;
5     private String str;
6
7     {
8         System.out.println(x: "initializing number!");
9         number = 100;
10        str = "";
11    }
12
13    public CodeBlocks(){
14        System.out.println(x: "Constructor 1 of CodeBlocker is running -->");
15    }
16
17    public CodeBlocks(int num){
18        this.number = num;
19        System.out.println(x: "Constructor 2 of CodeBlocker is running -->");
20    }
21 }
```

静态构造代码块只在类第一次加载时执行一次，之后不再执行，而非静态代码在每次new时执行一次

```

3 public class CodeBlocks {
4     private int number;
5     private static int num;
6     private String str;
7
8     {
9         System.out.print(s: "code block -->");
10        number = 100;
11        str = "";
12    }
13
14    static {
15        System.out.print(s: "static code block -->");
16        num = 10;
17    }
18
19    public CodeBlocks(){

```

执行顺序

静态代码块—>非静态代码块—>构造方法

example

CodeBlocks

== first new CodeBlocks ==
 static code block -->
 code block -->
 constructor 1 of CodeBlocker -->
 normal function
 ==second new CodeBlocks ==
 code block -->
 constructor 1 of CodeBlocker -->
 normal function

```

3 public class CodeBlocks {
4     private int number;
5     private static int num;
6     private String str;
7
8     {
9         System.out.print(s: "code block --> ");
10        number = 100;
11        str = "";
12    }
13
14    static {
15        System.out.print(s: "static code block --> ");
16        num = 10;
17    }
18
19    public CodeBlocks(){
20        System.out.print(s: "Constructor 1 of CodeBlocker --> ");
21    }
22
23    public void test(){
24        System.out.println(x: "normal function");
25    }
26 }

```

```

System.out.println(x: "===== first new CodeBlocks =====");
CodeBlocks cb = new CodeBlocks();
cb.test();
System.out.println(x: "===== second new CodeBlocks =====");
CodeBlocks cb2 = new CodeBlocks();
cb2.test();

```

航空航天大学

Lecture 4 继承 (inheritance)

[Java中static静态类和静态方法隐藏、重写、继承](#)

[Java为什么只能单继承?](#)

继承的优点

- 代码的可重用性
- 父类的属性和方法可用于子类
- 设计应用程序变得更加简单
- 可以轻松地自定义子类

基类和派生类的关系

- 基类是对若干个派生类的抽象
 - 基类抽取了派生类的公共特征，在设计时，应该将通用的方法放到超类中
 - Java中所有类共同的基类是**Object**类
- 派生类是基类的具体化
 - 通过扩展超类定义子类的时候，仅需指出子类和超类的不同之处，**通过增加数据成员或成员函数将基类变为某种更有用的类型**
- **派生类可以看作基类定义的延续**

this调用构造函数

1. 引用自身对象的成员变量 e.g. `this.age;`
2. 引用自身对象的成员方法 e.g. `this.display();`
3. 调用自身的构造方法 e.g. `this("Jack", Male, 10);`

(在有参构造函数中先调用无参构造函数)

```
public class Animal{
    String name;
    protected String type = "Animal";
    public Animal(){
        System.out.println(x: "Animal's Constructor 1 is running!");
    }
    public Animal(String name){
        this();
        this.name = name;
        System.out.println(x: "Animal's Constructor 2 is running!");
    }
}
```

super关键字

1. 引用父类对象的成员变量 e.g. `super.age;`
2. 引用父类对象的成员方法 e.g. `super.display();`
3. 调用父类的构造方法 e.g. `super("Jack", Male, 10);`


```
public class Deer extends Animal {
    public Deer(String name){
        super(name);
        System.out.println(x: "Deer's constructor 1 is running");
    }
}
```

this引用和super引用的对比

Java每个类都默认地具有 `null`、`this`、`super` 三个域，所以在任何类中都可以不加说明就可以直接引用它们

子类构造函数

- 无参子类构造函数
 - **显式调用**：使用 `super` 关键字进行调用父类的构造函数
 - **隐式调用**：不使用 `super` 进行调用则默认进行父类**无参数构造函数**的调用，即 `super.ObjectName()`
- 有参子类构造函数
 - 无论使用 `this` 调用本类的构造函数，还是使用 `super` 调用父类的构造函数，**都必须是该方法体中的第一条可以执行语句，否则会产生语法错误**

```
public Deer(String name){
    System.out.println(x: "Deer's constructor is running");
    super(name);
}
```

子类对象的生成

(构建)

- 子类创建对象时，子类的构造方法总是先调用父类的某个构造方法，完成父类部分的创建；然后再调用子类自己的构造方法，完成子类部分的创建。
- 如果子类的构造方法没有明显地指明使用父类的哪个构造方法，子类就调用父类的不带参数的构造方法。
- 子类在创建一个子类对象时，不仅子类中声明的成员变量被分配了内存，而且父类的所有的成员变量也都分配了内存空间。

(调用顺序)

- 创建子类对象时，子类总是按层次结构从上到下的顺序调用所有超类的构造函数。
- 如果父类没有不带参数的构造方法，则在子类的构造方法中必须明确的告诉调用父类的某个带参数的构造方法，通过 `super` 关键字，这条语句还必须出现在构造方法的第一句。

example

```

public class Animal{
    String name;
    public Animal(){
        System.out.println(x: "Constructor 1 is running!");
    }
    public Animal(String name){
        this.name = name;
        System.out.println(x: "Constructor 2 is running!");
    }
}

```

```

public class Deer extends Animal {
    public Deer(String name){
        super(name);
        System.out.println(x: "Deer's constructor 1 is running");
    }
    public Deer(String name, int age){
        this(name);
        System.out.println(x: "Deer's constructor 2 is running");
    }
    public void eat(){
        System.out.print
    }
    public String toString(){
        return "My name
    }
}

class Main{
    Run | Debug
    public static void main(String[] args){
        Deer deer = new Deer(name: "little deer");
        System.out.println(deer);
    }
}

```

Animal's Constructor 2 is running!
Deer's constructor 1 is running
My name is little deer

```
public class Deer extends Animal {
    public Deer(String name){
        super(name);
        System.out.println(x: "Deer's constructor 1 is running");
    }
    public Deer(String name, int age){
        this(name);
        System.out.println(x: "Deer's constructor 2 is running");
    }
    public void eat(){
        System.out.print
    }
    public String toString(){
        return "My name"
    }
}

class Main{
    Run | Debug
    public static void main(String[] args){
        Deer deer = new Deer(name: "little deer", age: 2);
        System.out.println(deer);
    }
}
```

24/ 47

Animal's Constructor 2 is running!
Deer's constructor 1 is running
Deer's constructor 2 is running
My name is little deer

```
public class Deer extends Animal {
    public Deer(String name){
        super(name);
        System.out.println(x: "Deer's constructor 1 is running");
    }
    public Deer(String name, int age){
        System.out.println(x: "Deer's constructor 2 is running");
    }
    public void eat(){
        System.out.println(x: "我在吃草");
    }
    public String toString(){
        return "My name"
    }
}

class Main{
    Run | Debug
    public static void main(String[] args){
        Deer deer = new Deer(name: "little deer", age: 2);
        System.out.println(deer);
    }
}
```

44/ 48

Animal's Constructor 1 is running!
Deer's constructor 2 is running
My name is little deer

protected 修饰符

- 父类的 protected 成员是包内可见且子类可见

- 若子类 and 父类不在同一包中，那么在子类中，**子类实例可以访问其从父类继承而来的 `protected` 方法，而不能访问父类实例的 `protected` 方法**（相当于就是这个父类实例和继承父类没有关系了，就是一个不同包、无继承的实例）

```
package cn.edu.buaa.oop.lesson_5.subpackage;
```

```
import cn.edu.buaa.oop.lesson_5.Parent;
```

```
public class Son2 extends Parent{
```

Run | Debug

```
    public static void main(String []args){
```

```
        Son2 son2 = new Son2();
```

```
        son2.message();
```

```
        son2.getMessage();
```

```
        Parent parent = new Parent();
```

```
        parent.getMessage();
```

```
    }
```

```
    private void message(){
```

```
        super.getMessage();
```

```
        System.out.println(x: "Son1's message");
```

```
    }
```

```
}
```

- 不同包下，**在子类中不能通过另一个子类引用访问共同基类的 `protected` 方法**（可以理解为这个新的子类的引用和当前的子类没有关系，完全是在另一个包内 `new` 出来的，其不属于子类内部继承出来的也不属于包内可访问的方法，所以会报错）

```

package cn.edu.buaa.oop.lesson_5.subpackage1;

import cn.edu.buaa.oop.lesson_5.Parent;

public class Son2 extends Parent{

    package cn.edu.buaa.oop.lesson_5.subpackage2;

    import cn.edu.buaa.oop.lesson_5.Parent;
    import cn.edu.buaa.oop.lesson_5.subpackage1.Son2;

    public class Son3 extends Parent{
        Run | Debug
        public static void main(String []args){
            Son2 son2 = new Son2();
            son2.getMessage();
        }
        private void message(){
            super.getMessage();
            System.out.println(x: "Son1's message");
        }
    }
}

```

- 对于 `protected` 修饰的**静态变量**，无论是否同一个包，在子类中均可直接访问（如例子中直接通过对象名称访问）

```

package cn.edu.buaa.oop.lesson_5;

public class Parent {
    private String privateField = "private field";
    protected String protectedField = "protected field";
    public String publicField = "public field";
    protected static String staticField = "static field";

    protected static void
        System.out.print
        System.out.print
}

package cn.edu.buaa.oop.lesson_5.subpackage1;

import cn.edu.buaa.oop.lesson_5.Parent;

public class Kid2 extends Parent{
    Run | Debug
    public static void main(String []args){
        System.out.println(Kid2.staticField);
        System.out.println(Parent.staticField);

        Kid2.staticField = "Kid2's static field";
        System.out.println(Kid2.staticField);
        System.out.println(Parent.staticField);
    }
}

```

static field
 static field
 Kid2's static field
 Kid2's static field

- 对于 `protected` 修饰的**静态函数**，无论是否同一个包，在子类中均可直接访问。在不同包的非子类中则不可访问

```

package cn.edu.buaa.oop.lesson_5.subpackage1;

import cn.edu.buaa.oop.lesson_5.Parent;

public class Kid2 extends Parent{
    Run | Debug
    public static void main(String []args){
        Parent parent = new Parent();
        parent.getMessage();
        Parent.getMessage();
    }
}

package cn.edu.buaa.oop.lesson_5.subpackage1;
import cn.edu.buaa.oop.lesson_5.Parent;

public class Kid4 {
    Run | Debug
    public static void main(String []args){
        Parent parent = new Parent();
        parent.getMessage();
        Parent.getMessage();
    }
}

```

继承中的 `public` 和默认访问级别

在Java中，类可以是 `public` 和默认访问级别

- `public` 级别的类可以被所有其他类继承
- 默认级别的类只能被同一个包中的类继承

向上转型

- 将子类转换成父类，在继承关系上是向上移动的，所以一般称为**向上转型**或者**向上映射**
- 由于向上转型是从一个**专用类型**向**通用类型**，所以**总是安全的**

变量隐藏

- 在子类对父类的继承中，如果**子类的成员变量和父类的成员变量同名**，此时称为**子类隐藏 (override / 重写) 了父类的成员变量**
- 子类若要引用父类的同名变量，要用 `super` 关键字做前缀

方法隐藏和方法覆盖

- **覆盖**：就是子类的方法和父类的方法具有完全一样的签名（名称、参数、返回类型完全相同）
- **隐藏**：**私有方法、静态方法不能被覆盖**，如果自来出现了同签名的方法，则为隐藏
- **final**：用 **final** 声明的成员方法是最终方法，不能被子类覆盖

区别：

- **覆盖**：调用的总是**实例化对象**的同名方法
- **隐藏**：调用的总是**引用的类型**的同名方法或同名变量

```
package com.buaa.test;

class Planet {
    public static void hide() {
        System.out.println("The hide method in Planet.");
    }
    public void override() {
        System.out.println("The overrid method in Planet.");
    }
}

public class Earth extends Planet {
    public static void hide() {
        System.out.println("The hide method in Earth.");
    }
    public void override() {
        System.out.println("The override method in Earth.");
    }
    public static void main(String[] args) {
        Earth myEarth = new Earth();
        Planet myPlanet = myEarth;
        myPlanet.hide();
        myPlanet.override();
    }
}
```

如何解决需要多继承的问题——钻石问题

使用组合


```
class ClassD{
```

```
    ClassA objA = new ClassA();
```

```
    ClassB objB = new ClassB();
```

```
    public void test(){  
        objA.doSomething();  
    }
```

```
    public void methodA(){  
        objA.methodA();  
    }
```

```
    public void methodB(){  
        objB.methodB();  
    }
```

- 组合能解决钻石问题
- 组合不会破坏类的封装
- 组合能精确控制父类子类变量方法的访问权限
- 组合会创建更多的对象, 占用内存



10

92

Lecture 5 多态

为什么使用多态

- 又要给XXX看病时，只需：
- 1. 编写XXX类继承Pet类（旧方案也需要）
- 2. 创建XXX类对象（旧方案也需要）
- 3. 其他代码不变（不用修改Doctor类）

代码可扩展性、可维护性变好了！

动多态：运行时根据所引用的具体实例来确定调用父类方法还是子类方法

- 同一个引用类型，使用不同的实例而执行不同的操作

```
public class Doctor {
```

```
    public void cure(Animal animal) {  
        if (animal.getHealth() < 50)  
            animal.toHospital();  
    }  
}
```

```
... ..
```

```
Animal dog = new Dog();  
Doctor doctor = new Doctor();  
doctor.cure(dog);  
Lion lion = new Lion();  
doctor.cure(lion);  
... ..
```

- **多态**是面向对象程序设计的另一个**重要特征**，其基本含义是“拥有多种形态”，具体值在程序中**用相同的名称来表示不同的含义**

为什么会出现多态

- Java中引用变量有两个类型：
 - 一个是**编译时的类型**
 - 一个是**运行时的类型**
- 如果编译时的类型与运行时的类型不一致，就会出现所谓的**多态**

多态性类型

- **静态多态**（静多态）：在编译时决定调用哪个方法，也成为**编译时多态**，也称为**静态联编**，也称为**静绑定**

静态多态一般是指**方法重载**

（因此，静态多态与是否发生继承没有必然联系）

- 对于**方法重载**，**返回值类型**与访问权限修饰符可以相同也可以不同，上述两项不能当做判断是否重载的条件。

- 如果一个类中有两个同名方法，其参数列表完全一样，仅仅返回值类型不同，则编译时会产生错误

- **动态多态**（动多态）：方法覆盖是子类的成员方法重写了父类的成员方法，重写的目的很大程度上是为了实现多态

动态多态即在运行时才能确定调用哪个方法，也成为**运行时多态**，也成为**动态联编**，也称为**动绑定**

- Java中，实现动态多态有3个条件：继承、覆盖、向上转型，缺一不可。
 - “覆盖(override)方法、抽象方法和接口” 和动态联编关系紧密

example

```
1  class Sup {
2      public int x, y;
3      Sup(int a, int b) {
4          x = a;
5          y = b;
6      }
7
8      public void display() {
9          int z;
10         z = x + y;
11         System.out.println("add=" + z);
12     }
13 }
14
15 class Sub extends Sup {
16     Sub(int a, int b) {
17         super(a, b);
18     }
19
20     public void display() {
21         int z;
22         z = x * y;
23         System.out.println("product=" + z);
24     }
25 }
```

```

public class ResultDemo extends Sub
{
    ResultDemo(int x,int y)
    {
        super(x,y);
    }
    public static void main(String args[ ])
    {
        Sup num1=new Sup(7,14);
        Sub num2=new Sub(7,14);
        ResultDemo num3=new ResultDemo(7,14);
        num1.display( );
        num2.display( );
        num3.display( );
        num1=num2;
        num1.display();
        num1=num3;
        num1.display();
    }
}

```

```

add=21
product=98
product=98
product=98
product=98

```

方法覆盖注意事项

- 子类方法不能缩小父类方法的访问权限
- **方法覆盖**：方法名、参数个数、参数类型及参数顺序必须一致（函数签名）；
- 子类方法不能缩小父类方法的访问权限：
 - a) 一个默认访问权限方法可以被重写为默认权限、protected和public的；
 - b) 一个protected方法可以被重写为protected和public的；
 - c) 一个public方法只可以被重写为public的；

```
public class Dog extends Animal {
    public Dog(String name) {
        super(name);
    }

    protected void toHospital() {
        this.setHealth(i: 60);
        System.out.println(x: "Curing Dog: 打针、吃药");
    }
}

public class Animal {
    String name;
    protected String type = "Animal";
    protected int healthValue = 99;

    public Animal(String name) {
        this.name = name;
    }

    public void toHospital() {
    }
}

void cn.edu.buaa.oop.lesson_6.Dog.toHospital()
Cannot reduce the visibility of the inherited method from
Animal Java(67109273)
```

多态技术的优点

- 引进多态技术之后，尽管子类的对象千差万别，但都可以采用 **父类引用.方法名([参数]) 统一** 方式来调用，在程序运行时能根据子对象的不同得到不同的结果。
- 应用程序不必为每一个派生类（子类）编写功能调用，只需要对抽象基类进行处理即可。这种“**以不变应万变**”的形式可以规范、简化程序设计，符合软件工程的“**一个接口，多种方法**”思想，可以**大大提高程序的可复用性**。
- 派生类的功能可以被基类的引用变量引用，这叫**向后兼容**，可以提高程序的**可扩充性和可维护性**。

为什么使用抽象类

- 如果父类方法没有机会被访问调用，或者没有办法给出明确的定义

```
public class Figure {  
    public void area();  
}
```

每个子类的area（）方法实现不同，父类area（）方法的实现是多余的。

- 因此，可以使用抽象方法来**优化**
抽象方法可以没有方法体

```
public abstract void area();
```

没有方法体

抽象类和抽象方法

- 关键字 `abstract` 可用来修饰方法和类，表示**尚未实现**的含义
- **抽象类**：如果一个类用 `abstract` 修饰，则它是一个**抽象类**
- **抽象方法**：如果一个方法用 `abstract` 修饰，则是一个**抽象方法**；抽象方法没有方法体，以一个紧跟在声明后的 `;` 结束

注意：无方法体与方法体为空是两个不同的概念。

抽象类特征

- **抽象方法**必须在**抽象类**中
- 如果一个类**继承**自某个**抽象父类**，而**没有具体实现**抽象父类中的抽象方法，则必须定义为**抽象类**
 - 抽象方法由子类实现，除非子类是抽象类
- 如果一个类里有**抽象方法**，则这个类必须声明称**抽象的**。但一个抽象类中**可以没有抽象方法**

```
public abstract class AbstractAnimal{
    String name;
    protected String type = "Animal";
    protected int healthValue = 99;

    public AbstractAnimal(String name){
        this.name = name;
    }

    public abstract void enjoy();

    public static void test(){

    }
}
```

注意!

- 抽象方法不能被private、final或static修饰。为什么？
 - ① 抽象方法必须被子类所覆盖，如果说明为private，则外部无法访问，覆盖也无从谈起。
 - ② 若说明为static，即使不创建对象也能访问：**类名.方法名()**，这要求给出方法体，但与抽象方法的定义相矛盾。
 - ③ Final和abstract含义矛盾

- **抽象类是不能实例化的**，但可以创建它的引用。它的作用是提供一个恰当的父类。因此一般作为其它类的超类，与final类正好相反。
- Java支持多态性，允许通过**父类引用**来引用子类的对象。

工厂模式

- **定义**：定义一个创建产品对象的工厂接口，将产品对象的实际创建工作推迟到具体子工厂类当中。满足**创建型模式**中所要求的**创建与使用相分离**的特点
 - 简单工厂模式
 - 工厂方法模式
 - 抽象工厂模式

example

(从上边的改成下边的)

```
public abstract class Operation {
    private double value1 = 0;
    private double value2 = 0;
    protected abstract double getResult();
}

//加法
public class OperationAdd extends Operation {
    @Override
    protected double getResult() {
        return getValue1() +
    }
}

//减法
public class OperationSub extends Operation {
    @Override
    protected double getResult() {
        return getValue1() -
    }
}

public static void main(String[] args) {
    //计算两数之和
    OperationAdd operationAdd = new OperationAdd();
    operationAdd.setValue1(1);
    operationAdd.setValue2(2);
    System.out.println("sum:"+operationAdd.getResult());
    //计算两数乘积
    OperationMul operationMul = new OperationMul();
    operationMul.setValue1(3);
    operationMul.setValue2(5);
    System.out.println("multiply:"+operationMul.getResult());
    //计算两数之差。。。
}
```

使用面向对象的形式
定义计算器

```
public class OperationFactory {  
    public static Operation createOperation(String operation) {  
        Operation oper = null;  
        switch (operation) {  
            case "add":  
                oper = new OperationAdd();  
                break;  
            case "sub":  
                oper = new OperationSub();  
                break;  
            case "mul":  
                oper = new OperationMul();  
                break;  
            case "div":  
                oper = new OperationDiv();  
                break;  
            default:  
                throw new UnsupportedOperationException("不支持该操作");  
        }  
        return oper;  
    }  
}
```

抽象产品类

具体产品类

```
public static void main(String[] args) {  
    Operation operationAdd = OperationFactory.createOperation("add");  
    operationAdd.setValue1(1);  
    operationAdd.setValue2(2);  
    System.out.println(operationAdd.getResule());  
}
```

工厂模式优缺点

(优点)

- 可以使代码结构清晰，有效地封装变化。在编程中，产品类的实例化有时候是比较复杂和多变的，通过工厂模式，将产品的实例化封装起来，使得调用者根本无需关心产品的实例化过程，只需依赖工厂即可得到自己想要的产品。
- 对调用者屏蔽具体的产品类。如果使用工厂模式，调用者只关心产品的接口就可以了，至于具体的实现，调用者根本无需关心。即使变更了具体的实现，对调用者来说没有任何影响。
- 降低耦合度。产品类的实例化通常来说是很复杂的，它需要依赖很多的类，而这些类对于调用者来说根本无需知道，如果使用了工厂方法，我们需要做的仅仅是实例化好产品类，然后交给调用者使用。对调用者来说，产品所依赖的类都是透明的。

(存在的问题)

- 加入新产品的时候违背了开闭原则
 - 简单工厂模式又叫作静态工厂方法模式 (Static Factory Method Pattern)
 - 当我们需要增加一种计算时，例如开平方。这个时候我们需要先定义一个类继承 Operation 类，其中实现平方的代码。除此之外我们还要修改 OperationFactory 类的代码，增加一个case。这显然是违背开闭原则的。可想而知对于新产品的加入，工厂类是很被动的。

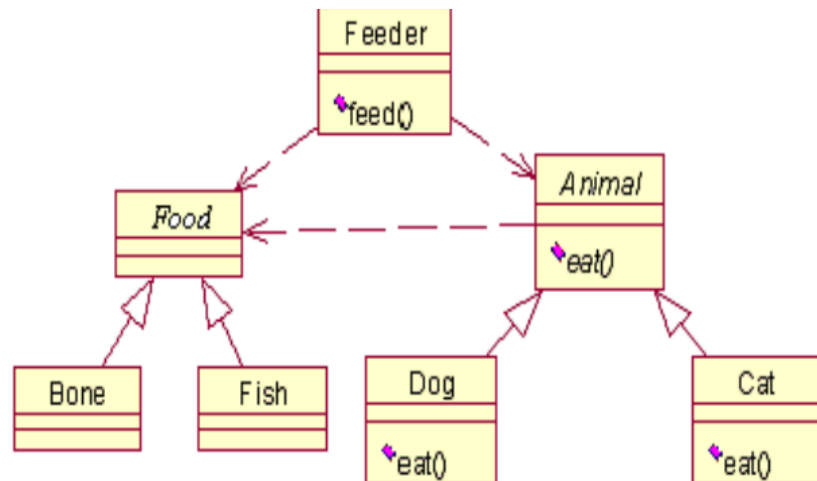
(因此孕育了抽象工厂模式)

多态案例试题分析

- 在这个动物园里，涉及的对象有
 - 饲养员
 - 各种不同动物
 - 以及各种不同的食物。
- 当饲养员给动物喂食时，动物发出欢快的叫声。
- 很容易抽象出3个类**Feeder**、**Animal**和**Food**。
- 假设只考虑猫和狗，则由Animal类派生出Cat类和Dog类
- 由Food类可以进一步派生出其子类Bone、Fish。因为他们之间存在着明显的is-a关系。

用到的知识点

- 继承
- 多态
- 抽象类



```
package com.buaa.adstractAnimal;  
public abstract class Food {  
    public abstract String getName();  
}
```

```
package com.buaa.adstractAnimal;  
public abstract class Animal {  
    private String name;  
  
    public Animal(String name) {  
        this.name = name;  
    }  
    public abstract void eat(Food food);  
    public String getName() {  
        return this.name;  
    }  
}
```

```
package com.buaa.adstractAnimal;  
public class Cat extends Animal {
```

```
    public Cat(String name) {  
        super(name);  
    }
```

```
    @Override  
    public void eat(Food food) {  
        System.out.println(  
            "小猫" + this.getName()  
            + "正在吃着" + food.getName()  
        );  
    }  
}
```

```
package com.buaa.adstractAnimal;
public class Feeder {

    private String name;
    public Feeder(String name) {
        this.name = name;
    }

    public void feed(Animal animal, Food food) {
        System.out.println(
            "饲养员" + this.name
            + "喂养动物" + animal.getName()
        );
        animal.eat(food);
    }
}
```

Lecture 6 接口

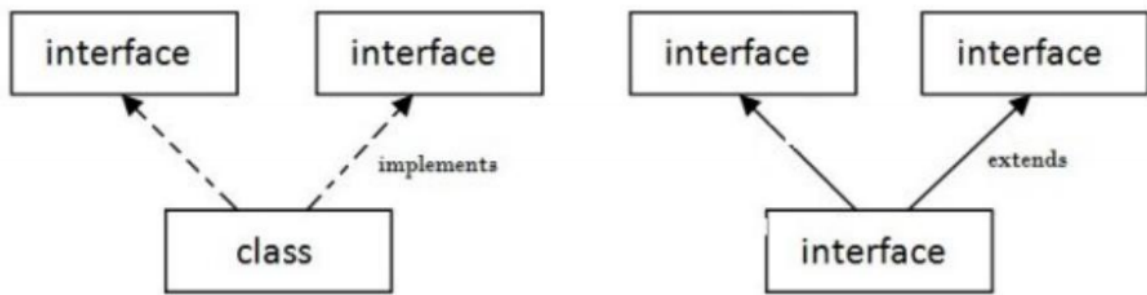
[接口中的变量默认是public static final的](#)

接口的含义

(更多的时候是一个**形容词**或者一个**动词**)

- ① 一是指概念性的接口，即指类对外提供的所有服务。
类的所有能被其他程序访问的方法构成了类的接口。
- ② 二是指用 **interface关键字** 定义的实实在在的接口，
也称为 **接口类型**。Java接口是一系列方法的声明，是一些方法特征的集合
- ③ 一个接口只有方法的特征没有方法的实现，因此这些方法可以在不同的地方被不同的类实现，而这些实现可以具有不同的行为（功能）

接口和类、接口和接口的关系



接口的必要性

- 通过接口可以实现不相关类的相同行为，而不需要考虑这些类之间的层次关系
- 通过接口可以了解对象的**交互界面**，而不需要了解内部细节
- 1、**丰富Java面向对象的思想**：在Java语言中，abstract class 和interface 是支持抽象类定义的两机制。正是由于这两种机制的存在，才赋予了Java强大的面向对象能力。
- 2、**有利于代码的规范性**：如果一个项目比较庞大，那么就需要一个能理清所有业务的架构师来定义一些主要的接口，这些接口不仅告诉开发人员你需要实现那些业务，而且也将命名规范限制住了
- 3、**有利于对代码进行维护**：可以一开始定义一个接口，把功能菜单放在接口里，然后定义类时实现这个接口，以后要换的话只不过是引用另一个类而已，这样就达到维护、拓展的方便性。
- 4、**增强安全、严密性**：接口是实现软件松耦合的重要手段，它描叙了系统对外的所有服务，而不涉及任何具体的实现细节。这样就比较安全、严密一些(一般软件服务商考虑的比较多)。

定义接口

- 即接口中的**成员变量**均是默认 `public static final`
- 方法均需要是 `public abstract`

```
[public] interface 接口名称[extends 父接口名列表]
{
    //静态常量
    [public][static][final]数据类型 变量名=常量名;
    //抽象方法
    [public][abstract][native]返回值类型
                                方法名（参数列表）;
}
```

实现接口

- 和继承抽象类类似，实现一个接口需要**实现接口中定义的所有方法**
- 一个类在实现某个接口的方法时，必须以**完全相同的方法头（方法签名）**。否则，只是在**重载一个新方法**，而不是实现已有的方法
- 只**实现部分方法**时，需要将类设置为**抽象类**

```
[修饰符]class 类名[extends 父类名] [implements 接口A, 接口B, ...]
{
    类的成员变量和成员方法;
    为接口A中的所有方法编写方法体，实现接口A;
    为接口A中的所有方法编写方法体，实现接口B;
    ...
}
```

```
[修饰符] class A implements IA {...}
[修饰符] class B extends A implements IB,
IC {...}
```

```
interface IExample {  
    void method1();  
    void method2();  
}
```

```
abstract class Example1 implements IExample {  
    public void method1() {  
        //... ..  
    }  
}
```

因为只实现了一个方法，所以类Example1需要定义成抽象类。

实现多个接口

- 不会出现歧义

```
interface InterA {  
    public void doSomething();  
}
```

```
interface InterB {  
    public void doSomething();  
}
```

```
class MultiInterface implements InterA, InterB {  
    public void doSomething(){  
        //实现接口的方法  
    }  
}
```

接口中变量的访问

```
interface InterA {
    int field = 6;
    public abstract void doSomething();
}
```

默认 public static final

```
class InterfaceTest implements InterA {
    @Override
    public void doSomething() {
    }
}
```

Run | Debug

```
public static void main(String []args){
    InterfaceTest it = new InterfaceTest();
    System.out.println(it.field);
    System.out.println(InterfaceTest.field);
    System.out.println(InterA.field);
}
```

```
interface InterA {
    int field = 6;
    public abstract void doSomething();
}
```

```
class InterfaceTest implements InterA {
    int field = 5;
    @Override
    public void doSomething() { }
}
```

Run | Debug

```
public static void main(String []args){
    InterfaceTest it = new InterfaceTest();
    System.out.println(it.field);
    // System.out.println(InterfaceTest.field); //Compilation Error
    System.out.println(InterA.field);

    InterA ia = it;
    System.out.println(it.field);
    System.out.println(ia.field);
}
```

5
6
5
6

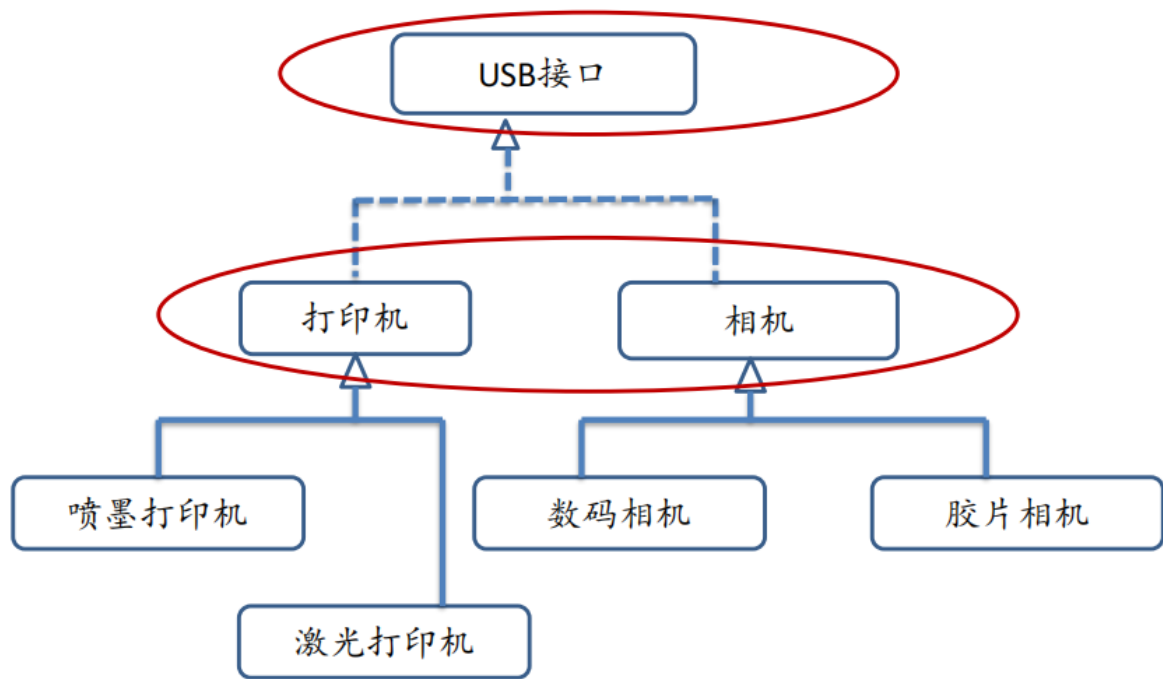
编译错误

抽象类和接口共同点

- 都不能被实例化
- **在语义上，都位于系统的抽象层，需要其他类来进一步提供实现细节。**
- 抽象类与接口都是为了继承与多态，它们都需要子类来继承或实现才有意义，最终目的是为了多态；
- 子类重写了父类的方法，再通过向上转型，由父类对象引用指向子类对象，达到运行时动态调用子类方法的目的。

抽象类和接口的区别

- 接口中的成员变量和方法只能是public类型的，而抽象类中的成员变量和方法可以处于各种访问级别。
- 接口中的成员变量只能是public、static和final类型的，而在抽象类中可以定义各种类型的实例变量和静态变量。
- 接口中没有构造方法，抽象类中有构造方法。接口中所有方法都是抽象方法，抽象类中可以有，也可以没有抽象方法。抽象类比接口包含了更多的实现细节。
- 抽象类是某一类事物的一种抽象，而接口不是类，它只定义了某些行为；
 - 例如，“生物”类虽然抽象，但有“狗”类的雏形，接口中的run方法可以由狗类实现，也可以由汽车实现。
- **在语义上，接口表示更高层次的抽象，声明系统对外提供的服务。**而抽象类则是各种具体类型的抽象。



接口回调

(和向上转向之后仍然调用引用的方法一个效果)

• 接口变量：

- 声明格式： 接口 变量名(又称为引用)
- 接口做参数：如果一个方法的参数是接口类型，就可以将任何实现该接口的类的实例的引用传递给接口参数，那么接口参数就可以回调类实现的接口方法。
- 把实现某一接口的类创建的对象引用赋给该接口声明的接口变量
- 该接口变量就可以调用被类实现的接口中的方法。
- 即：

接口变量=实现该接口的类所创建的对象；
接口变量.接口方法([参数列表]);

接口的进化

```
interface IA {...}
interface IB {...}
interface IC {...}
interface ID extends IA, IB, IC {...}
```

接口的意义

- 接口不仅是一种规范，而是 Java 变成思想的体现

父类和子类的对象之间转换

- 自动转换（向上映射）
- 强制类型转换（向下映射）
- 对象的向上映射总是安全的，可靠的。
- 对象的向下映射就不一定了，有时可以，有时不可以，**如果不可以转的话，程序是不会报语法错误的，发生的是一个运行时异常。**

上转型对象的使用

- 一. 上转型对象可以访问子类继承或隐藏的**成员变量，也可以调用子类继承的方法或子类重写的实例方法。**
- 二. 如果子类重写了父类的某个实例方法后，**当用上转型对象调用这个实例方法时一定是调用了子类重写的实例方法。**
- 三. 上转型对象不能操作子类新增的成员变量；不能调用子类新增的方法。

下转型对象的使用

- 向下转型(映射): 一个已经向上转型的子类对象可以使用强制类型转换的格式, 将父类引用转为子类引用, 这个过程是向下转型。
- 使用格式:
 - **子类类型 变量名 = (子类类型) 父类类型的变量;**
 - 如: `Person p = new Student();`
`Student stu = (Student) p`
 - **如果是直接创建父类对象, 是无法向下转型的! , 会产生运行时异常**
 - 如: `Person p = new Peron();`
`Student stu = (Student) p`

`instanceof` 操作符

- `instanceof`操作符用于判断一个引用类型所引用的对象是否是一个类的实例。`instanceof`运算符是Java独有的双目运算符
- `instanceof`操作符左边的操作元**是一个引用类型的对象**, 右边的操作元是一个类名或接口名。
- 形式如下:
 - `obj instanceof ClassName`**
 - 或者
 - `obj instanceof InterfaceName`**

Fish fish=new Fish();

//XXX表示一个类名或接口名

System.out.println(fish instanceof XXX);

- 当“XXX”是以下值时，instanceof表达式的值为**true**:
 - Fish类。
 - Fish类的直接或间接父类。
 - Fish类实现的接口。
- instanceof是Java中的二元运算符
- 表达式 **obj instanceof T**，instanceof 运算符的 **obj** 操作数的类型必须是引用类型; 否则，会发生编译时错误。
- 如果关系表达式的 instanceof 产生编译时错误, 则 **obj** 强制转换为 T 时同样发生编译错误。
- 如果 obj 不为 null 并且 (T) obj 不抛 ClassCastException 异常则该表达式值为 true , 否则值为 false 。

Lecture 7 面向对象的设计原则

高内聚 - 低耦合

- **低耦合**：是指软件系统中，模块与模块之间的直接依赖程度低。
- 模块之间存在依赖,模块独立性越差，耦合越强,导致一个模块的改动可能会影响到其他模块。**比如模块A直接操作了模块B中数据，则视为强耦合，若A只是通过数据与模块B交互，则视为弱耦合。**
- **好处**：独立的模块便于扩展,维护,写单元测试,如果模块之间重重依赖,会极大降低开发效率，代码可维护性差。
- **高内聚**：系统存在AB两个模块儿进行交互，如果修改了A模块儿不影响B模块的工作，那么认为A模块儿有足够的内聚。
- 一个模块应当尽可能独立完成某个功能。模块内部的元素,关联性越强,则内聚越高,模块单一性更强。
- **危害**：低内聚的模块代码,不管是维护,扩展还是重构都相当麻烦,难以下手。

面向对象的基本原则

设计原则	一句话归纳	目的
开闭原则	对扩展开放，对修改关闭	降低维护带来的新风险
依赖倒置原则	高层不应该依赖低层，要面向接口编程	更利于代码结构的升级扩展
单一职责原则	一个类只干一件事，实现类要单一	便于理解，提高代码的可读性
接口隔离原则	一个接口只干一件事，接口要精简单一	功能解耦，高聚合、低耦合
迪米特法则	不该知道的不要知道，一个类应该保持对其它对象最少的了解，降低耦合度	只和朋友交流，不和陌生人说话，减少代码臃肿
里氏替换原则	不要破坏继承体系，子类重写方法功能发生改变，不应该影响父类方法的含义	防止继承泛滥
合成复用原则	尽量使用组合或者聚合关系实现代码复用，少使用继承	降低代码耦合

- 开闭原则

- 所谓 开-闭原则 (Open-Closed Principle) 就是让你的设计应当对扩展开放，对修改关闭。
- 在给出一个设计时，应当首先考虑到**用户需求的变化**，将应对用户变化的部分设计为对扩展开放，而设计的核心部分是经过精心考虑之后确定下来的基本结构，这部分应当是对修改关闭的，即不能因为用户的需求变化而再发生变化，因为这部分不是用来应对需求变化的

- 单一职责原则

- **单一职责原则**: 一个类只负责一个功能领域中的相应职责。
- 在程序设计时，我们应该将程序功能最小化，每个函数只干一件事。

- 里氏代换原则

- **里氏代换原则**: 所有引用基类（父类）的地方必须能透明地使用其子类的对象。
- 任何**基类**可以出现的地方，子类一定可以出现。
- 里氏代换原则是继承复用的基石，只有当衍生类可以替换掉基类，软件单位的功能不受到影响时，基类才能真正被复用
- 子类继承父类时，除添加新的方法完成新增功能外，尽量不要重写父类的方法
- 实现“开-闭”原则的关键步骤就是抽象化。里氏代换原则是对实现抽象化的具体步骤的规范。

- 合成复用原则

- **合成复用原则**：尽量使用组合或者聚合关系实现代码复用，少使用继承。
- 如果要使用继承关系，则必须严格遵循里氏替换原则。合成复用原则同里氏替换原则相辅相成的，两者都是开闭原则的具体实现规范。

复用

- 组合对象来复用方法的优点是：
 - **对象组合是类继承之外的另一种复用选择**。新的更复杂的功能可以通过组合对象来获得。
 - 对象组合要求对象具有良好定义的接口。这种复用风格被称为**黑箱复用(black-box reuse)**，因为被组合的对象的内部细节是不可见的,对象只以"**黑箱**"的形式出现|。
- **对象与所包含的对象属于弱耦合关系**，因为，如果修改当前对象所包含的对象的类的代码，不必修改当前对象的类的代码。
- 当前对象可以在**运行时刻动态指定所包含的对象**。

多用组合少用继承

- 之所以提倡**多用组合，少用继承**，是因为在许多设计中，人们希望系统的类之间**尽量是低耦合的关系，而不希望是强耦合关系**。即在许多情况下需要避开继承的缺点，而需要组合的优点。
- 怎样合理地使用组合，而不是使用继承来获得方法的复用需要经过一定时间的认真思考、学习和编程实践才能悟出其中的道理。
- **依赖倒转原则**

- **依赖倒转原则**: 抽象不应该依赖于细节, 细节应当依赖于抽象。换言之, 要针对接口编程, 而不是针对实现编程。
- 通过抽象使各个类或模块的实现彼此独立, 不互相影响, 实现模块间的松耦合
- 面向接口、抽象类编程

面向接口编程

- **面向接口的编程**: 是面向对象设计 (OOD) 的非常重要的基本原则之一。
- 它的含义是: 使用接口和同类型的组件通讯, 即, 对于所有完成相同功能的组件, 应该抽象出一个接口, 它们都实现该接口。
- 当设计一个类时, 不让该类面向具体的类, **而是面向抽象类或接口**, 即所设计类中的重要数据是抽象类或接口声明的变量, 而不是具体类声明的变量
- 具体到JAVA中, 可以是**接口 (interface)**, 或者是**抽象类 (abstract class)**, 所有完成相同功能的组件都实现该接口, 或者从该抽象类继承。
- 客户代码只应该和该接口通讯。
- **接口隔离原则**
 - **接口隔离原则**: 使用多个专门的接口, 而不使用单一的总接口, 即客户端不应该依赖那些它不需要的接口。
- **迪米特法则**

- **迪米特法则**: 一个软件实体应当尽可能少地与其他实体发生相互作用。
- 迪米特法则又叫**最少知道原则**, 即一个类对自己依赖的类知道的越少越好。
- 对于被依赖的类不管多么复杂, 都尽量将逻辑封装在类的内部。对外除了提供public方法, 不对外泄露任何信息

对象的 `equals()` 方法与操作符 `==`

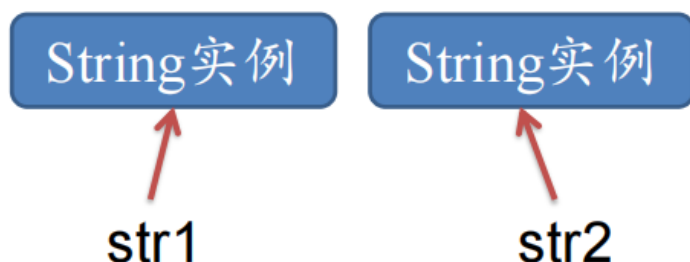
- 在JDK中有一些类覆盖了Object类的equals()方法, 它们的比较规则为: **如果两个对象的类型一致, 并且内容一致, 则返回true。**
- 这些类包括:
 - **java.io.File、**
 - **java.util.Date、**
 - **java.lang.String、**
 - **包装类 (如java.lang.Integer和java.lang.Double类等)。**

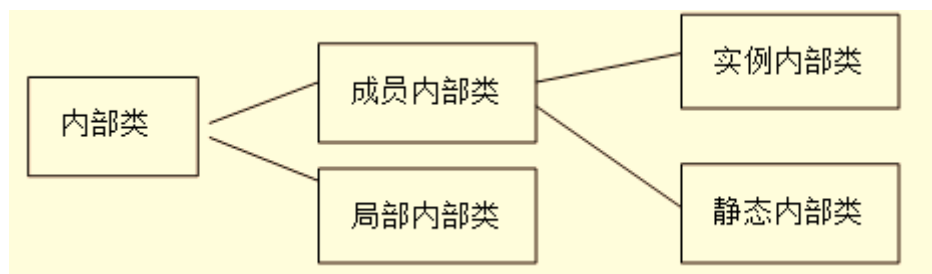
```
String str1=new String("Hello");
```

```
String str2=new String("Hello");
```

```
System.out.println(str1==str2); //打印false
```

```
System.out.println(str1.equals(str2)); //打印true
```





内部类

- 顶层类只能处于public和默认访问级别
- 而成员内部类可以处于**public、protected、默认和private**四个访问级别。

成员内部类

创建

```
class Outer {  
    public class InnerTool { // 内部类  
        public int add(int a, int b) {  
            return a + b;  
        }  
    }  
    private InnerTool tool = new InnerTool();  
  
    public void add(int a, int b, int c) {  
        tool.add(tool.add(a, b), c);  
        System.out.println(tool.add(tool.add(a, b), c));  
    }  
}
```

```
public class Tester {  
    public static void main(String args[]) {  
        Outer o = new Outer();  
        o.add(1, 2, 3);  
        Outer.InnerTool tool = new Outer().new InnerTool();  
    }  
}
```

在创建实例内部类的实例时，外部类的实例必须已经存在

性质

- 在内部类中，可以直接访问外部类的所有成员，包括成员变量和成员方法。
- 实例内部类的实例自动持有外部类的实例的引用。

静态内部类

- 静态内部类的实例不会自动持有外部类的特定实例的引用，在创建内部类的实例时，不必创建外部类的实例。
- 例如以下类A有一个静态内部类B，客户类Tester创建类B的实例时不必创建类A的实例：

```
class AA {  
    public static class B {  
        int v;  
    }  
}  
  
class Tester {  
    public void test() {  
        AA.B b = new AA.B();  
        b.v = 1;  
    }  
}
```

- 客户类可以通过完整的类名直接访问静态内部类的静态成员。

```
public class AAA {  
    public static class B {  
        int v1;  
        static int v2;  
        public static class C {  
            static int v3;  
            int v4;  
        }  
    }  
}
```

```
public class TesterAAA {  
    public void test() {  
        AAA.B b = new AAA.B();  
        AAA.B.C c = new AAA.B.C();  
        b.v1 = 1;  
        b.v2 = 1;  
        // AAA.B.v1=1; //编译错误  
        AAA.B.v2 = 1; // 合法  
        AAA.B.C.v3 = 1; // 合法  
    }  
}
```

局部内部类

- **注意：**只能访问方法中有 `final` 修饰的最终变量或者参数

- 局部内部类只能在当前方法中使用。
- 局部内部类和实例内部类一样，可以访问外部类的所有成员
- 此外，局部内部类还可以访问所在方法中的符合以下条件之一的参数和变量：
 - 最终变量或参数：用final修饰

匿名类

- 匿名类就是没有名字的类，是将类和类的方法定义在一个表达式范围里。
- 匿名类本身没有构造方法，但是会调用父类的构造方法。
- 匿名内部类将内部类的定义与生成实例的语句合在一起，并省去了类名以及关键字“class”，“extends”和“implements”等。

```
父类/接口 a = new 父类/接口(参数) {  
    方法.....  
}
```

```

public class AAAAA {
    AAAAA(int v) {
        System.out.println("another constructor");
    }
    AAAAA() {
        System.out.println("default constructor");
    }
    void method() {
        System.out.println("from AAAAA");
    };
    public static void main(String args[]) {
        new AAAAA().method(); // 打印from AAAAA
        AAAAA a = new AAAAA() { // 匿名类
            void method() {
                System.out.println("from anonymous");
            }
        };
        a.method(); // 打印from anonymous
    }
}

```

```

default constructor
from AAAAA
default constructor
from anonymous

```

2024/10/31

Xiang Gao

内部类的用途

1. 封装类型

- 如果一个类只能有系统中的某一个类访问，可以定义为该类的内部类。
- 此外，如果一个内部类仅仅为特定的方法提供服务，那么可以把这个内部类定义在方法之内。
- 可见，**内部类是一种封装类型的有效手段。**

2. 直接访问外部类的成员

- 内部类的一个特点是能够访问外部类的各种访问级别的成员。
- 假定有类A和类B，类B的**reset()**方法负责重新设置类A的实例变量**count**的值。
 - 一种实现方式是把类A和类B都定义为外部类：

3. 回调

- 在以下Adjustable接口和Base类中都定义了adjust()方法，这两个方法的参数签名相同，但是有着不同的功能。

```
public interface Adjustable{  
    /** 调节温度 */  
    public void adjust(int temperature);  
}  
  
public class Base{  
    private int speed;  
    /** 调节速度 */  
    public void adjust(int speed){  
        this.speed=speed;  
    }  
}
```

2024/10/31

Xiang Gao

 北京航空航天大学

- 如果有一个Sub类同时具有调节温度和调节速度的功能，那么Sub类需要继承Base类，并且实现Adjustable接口，但是以下代码并不能满足这一需求：

adjust(int speed)
调节速度

```
public class Sub extends Base implements Adjustable{  
    private int temperature;  
    public void adjust(int temperature){  
        this.temperature=temperature;  
    }  
}
```

adjust(int temerature)
调节温度

```

public class Sub extends Base {
    private int temperature;
    private void adjustTemperature(int temperature){
        this.temperature=temperature;
    }
    private class Closure implements Adjustable{
        public void adjust(int temperature){
            adjustTemperature(temperature);
        }
    }
    public Adjustable getCallbackReference(){
        return new Closure();
    }
}

```

2024/10/31

Xiang Gao



- 以下代码演示客户类使用Sub类的调节温度的功能：

```

public class TestClass {
    public static void main(String[] args){
        //调节温度
        Sub sub=new Sub();
        Adjustable ad=sub.getCallbackReference();
        ad.adjust(15);
        //调节速度
        sub.adjust(350);
    }
}

```

接口回调

- 回调实质上是指一个类尽管实际上实现了某种功能,但是没有直接提供相应的接口,客户类可以通过这个类的内部类的接口来获得这种功能。而这个内部类本身并没有提供真正的实现,仅仅调用外部类的实现。
- 可见,回调充分发挥了内部类具有访问外部类的实现细节的优势。

```
public class Sub extends Base {  
    private int temperature;  
  
    private void adjustTemperature(int temperature){  
        this.temperature=temperature;  
    }  
  
    private class Closure implements Adjustable{  
        public void adjust(int temperature){  
            adjustTemperature(temperature);  
        }  
    }  
    public Adjustable getCallBackReference(){  
        return new Closure();  
    }  
}
```

Lecture 9 Java异常处理 & Unit Test

异常概述

- 一. 编译系统检查出来的语法错误,导致程序运行结果不正确的逻辑错误,都不属于异常的范围。
- 二. 异常是一个可以正确运行的程序在运行中可能发生的错误。

异常处理

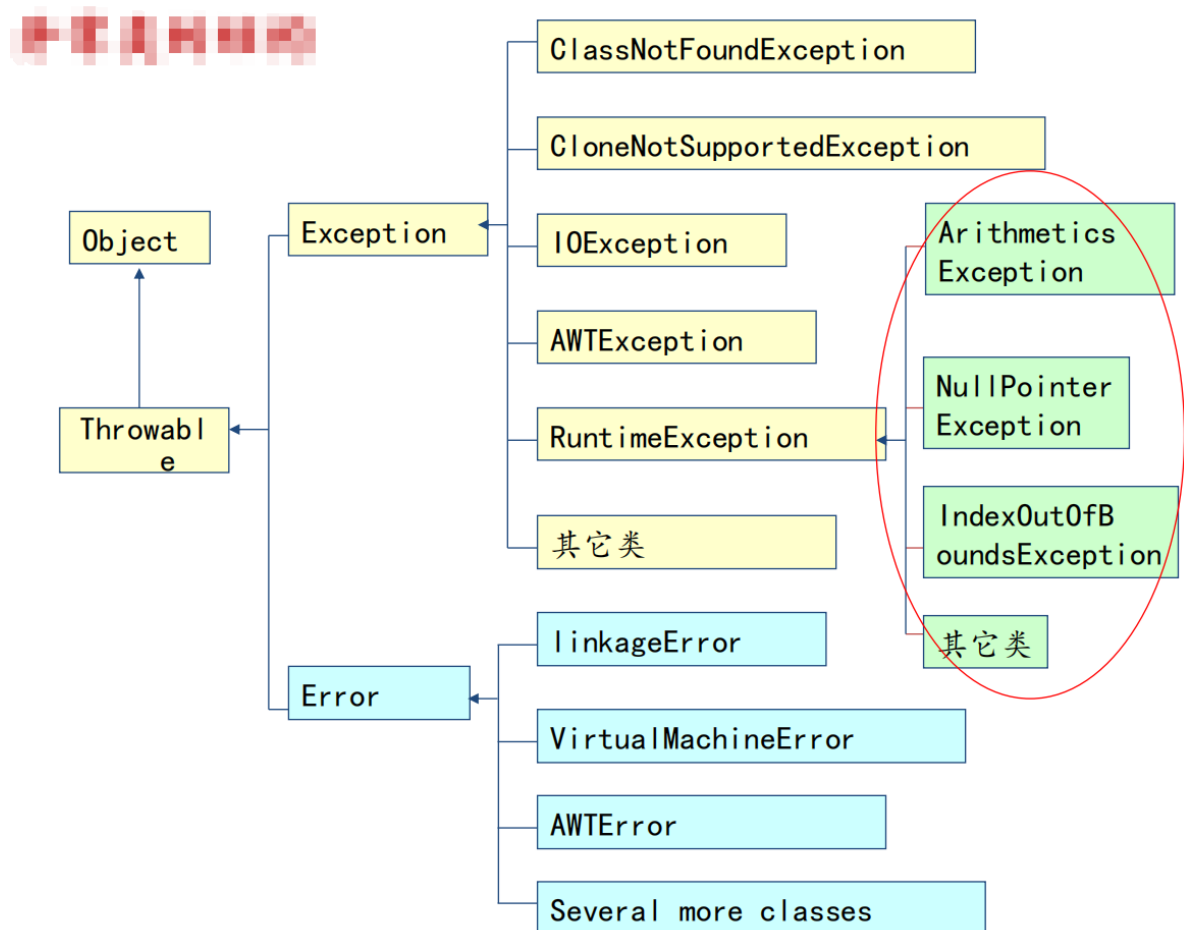
- 健壮性 (鲁棒性)

一. **异常处理 (Exception Handling)**：就是要提出或者是研究一种机制，能够较好的处理程序不能正常运行的问题。

二. 程序设计的要求之一就是程序的健壮性。希望程序在运行时能够不出或者少出问题。但是，在程序的实际运行时，总会有一些因素会导致程序不能正常运行。

例如文件没找到，网络错误，非法的参数，数组越界，空指针等。

异常类的结构



异常类的分类

- 一. Throwable是所有异常类的父类，它是Object的直接子类。是类库java.lang包中的一个类。
- 二. Exception类继承自Throwable类。所有的Throwable类的子孙类所产生的对象都是异常。
- 三. Error:由Java虚拟机生成并抛出, Java程序不做处理。
- 四. Runtime Exception:由系统检测，用户的Java程序可不做处理, 系统将它们交给缺省的异常处理程序。
- 五. **非Runtime Exception:Java编译器要求Java程序必须捕获（try, catch）或声明所有的非运行时异常。**

Java异常处理机制

- Java提供的是异常处理的**抓抛模型**。
- Java程序的执行过程中如出现异常，会自动生成一个**异常类对象**，该异常对象将被提交给Java运行时系统，这个过程称为**抛出(throw)异常**。

- 如果一个方法内抛出异常，该异常会被抛到调用方法中。如果异常没有在调用方法中处理，它继续被抛给这个调用方法的调用者。这个过程将一直继续下去，直到异常被处理。这一过程称为捕获(catch)异常。
- 如果一个异常回到main()方法，并且main()也不处理，则程序运行终止。
- Java语言按照面向对象的思想来处理异常：
 - 把各种不同类型的异常情况进行分类，用类来表示异常情况，这种类被称为异常类。
 - 用throw语句在方法声明处声明抛出特定异常，用throw语句在方法中抛出具体的异常
 - 用try-catch语句来捕获并处理异常。

处理异常的方法

- 如果方法中的代码块可能抛出异常，有两种处理办法：
 - (1) 在当前方法中通过try-catch语句捕获并处理异常
 - (2) 在方法的声明处通过throws语句声明抛出异常

异常处理的两种方式

try...catch...finally

throws语句：

```
public void methodA(int money){  
    try{  
        //以下代码可能会抛出SpecialException  
        if(--money<=0)  
            throw new SpecialException("Out of money");  
    }catch(SpecialException e){  
        处理异常  
    }  
}
```

```
public void methodA(int money) throws SpecialException{  
    //以下代码可能会抛出SpecialException  
    if(--money<=0)  
        throw new SpecialException("Out of money");  
}
```

finally语句

- 一般用于释放资源

一. 实际应用中，确保一段代码不管发生什么异常都能被执行是必要的，关键字 **finally** 就是用来标识这样一段代码的。

二. **Finally**: 无条件执行的语句，一般用于释放资源。即使没有 **catch** 子句，**finally** 语句块也会在执行了 **try** 语句块后立即执行。

```
try {  
    对文件进行处理的程序;  
} catch (IOException e) {  
    //对文件异常进行处理;  
} finally {  
    不论是否发生异常,都关闭文件;  
}
```

try-catch-finally结合

- 但要注意, 如果一个异常类和其子类都出现在 catch 子句中, **应把子类放在前面**, 否则永远不会到达子类, 会被父类直接抓取

```
try { //接受监视的程序块,在此区域内发生  
    //的异常,由catch中指定的程序处理;  
}  
catch (ExceptionType1 e) {  
    // 抛出ExceptionType1异常时要执行的代码  
}  
catch (ExceptionType2 e) {  
    // 抛出ExceptionType2异常时要执行的代码  
}.....  
[finally {  
    // //无条件执行的语句  
}]
```

Throws语句-声明异常

一. 声明异常：一个方法不处理它产生的异常，而是沿着调用层次向上传递，由调用它的方法来处理这些异常，叫声明异常。

二. Throws语句用来表明一个方法可能抛出的各种异常，并说明该方法会抛出但不捕获异常。

- 声明异常的格式：

<访问权限修饰符><返回值类型><方法名>(参数列表)
throws 异常列表{}

- 此时，这种异常交由调用此方法的方法进行处理，或交由系统进行处理。
- 如果最终方法也没有处理异常，异常将交给系统处理。

```
import java.io.FileInputStream;

public class ThrowsTest5 {
    public static void main(String args[]) throws IOException
    {
        FileInputStream in = new FileInputStream("myfile.txt");
        int b;
        b = in.read();
        while (b != -1) {
            System.out.print((char) b);
            b = in.read();
        }
        in.close();
    }
}
```

创建用户自定义异常类

- 语句形式:

```
<class><自定义异常名>extends<Exception>
{
    ...
}
```

- Throw抛出异常主要用于自定义的异常

一. 在前面讲述的例子中，异常对象是由Java在运行时由系统抛出的。

二. 程序中显示生成异常：抛出异常也可以通过代码来实现，throw语句可以明确的抛出一个异常。

三. Throw抛出异常主要用于自定义异常。

一. throw的语句格式为：

<throw><异常对象>

二. 程序会在throw语句处立即终止，转向try...catch 寻找异常处理方法。

`/** 表示车子出故障的异常情况 */`

```
public class CarWrongException extends Exception  
    public CarWrongException(){}  
    public CarWrongException(String msg){super(msg);}  
}
```

```
public class Car{  
    public void run()throws CarWrongException{  
        /*如果车子出故障，就创建一个CarWrongException  
        对象，并将其抛出*/  
        if(车子无法刹车)  
            throw new CarWrongException("车子无法刹车");  
        if(发动机无法启动)  
            throw new CarWrongException("发动机无法启动");  
    }  
}
```

throws语句在方法声明处声明抛出特定异常，
throw语句在方法中抛出具体的异常

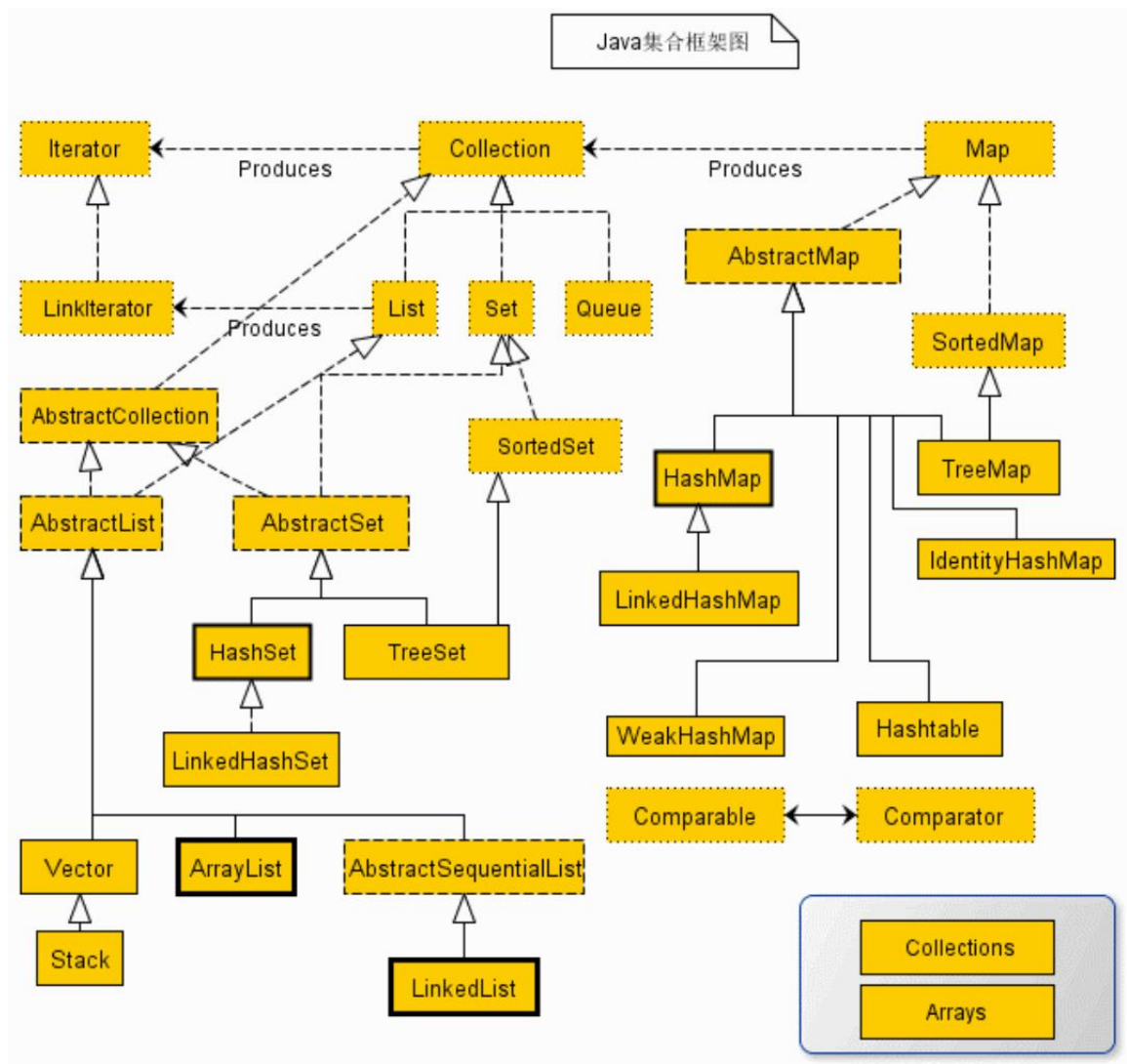
```
public class Worker{  
    private Car car;  
    public Worker(Car car){this.car=car;}  
    public void gotoWork()throws LateException{  
        try{  
            car.run();  
        }catch(CarWrongException e){  
            walk();  
            Date date=new Date(System.currentTimeMillis());  
            String reason=e.getMessage();  
            throw new LateException(date,reason); }  
    }  
    public void walk(){.....} //步行上班
```

异常处理原则

- 早处理
- 异常设计时需要分层次
- 只捕获特定异常：
 - 不要试图用一个catch块处理所有异常，也不要尝试把大量代码放在一个try语句中进行监测。这样不容易分析异常发生的位置，及分析异常的具体类型。
- 善用finally语句块
- Java 异常处理是使用 Java语言进行软件开发和测试脚本开发中非常重要的一个方面。**对异常处理的重视会使你开发出的代码更健壮，更稳定。**

Lecture 10 集合

Java集合框架图



副作用

在函数式编程中，输入一旦确定了，输出都确定了，函数调用的结果只依赖于传入的输入变量和内部逻辑，不依赖于外部，这样的写出的函数没有副作用

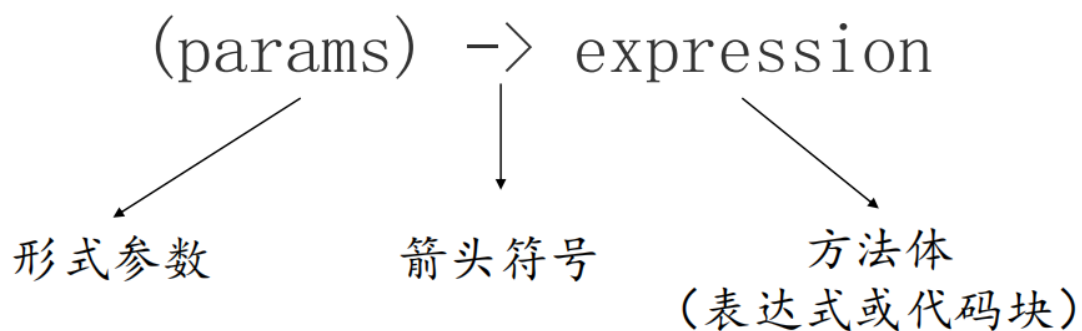
```
int a = 2;  
public int add(int b){  
    return a + b;  
}  
  
public int pow(int c){  
    return c * c;  
}
```

Side effect

2024/11/10



函数式编程



- Java提供的函数值接口 - `java.util.function`

(接口，亦可以接收lambda表达式传递的函数)

- `BiFunction`
- `Supplier`
- `Consumer`
- `Predicate`

```

BiFunction<Integer, Integer, Integer> f1 = (a, b) -> {
    int c = a + b;
    return c;
};
System.out.println("addition results: " + f1.apply(t: 3, u: 2));

BiFunction<Integer, Integer, Integer> f2 = (a, b) -> a * b;
System.out.println("multiply results: " + f2.apply(t: 3, u: 2));

Function<Integer, Integer> f3 = a -> a * a;
System.out.println("square results: " + f3.apply(t: 3));

```

addition results: 5
 multiply results: 6
 square results: 9

//数据的提供者， 只出不进。

```

Supplier<Integer> supplier = () -> new Random().nextInt();
System.out.println("supplier: " + supplier.get());

```

//数据的消费者， 只进不出

```

Consumer<Integer> consumer = (Integer x) -> {
    System.out.println("consumer value " + x);
};
consumer.accept(t: 5);

```

//判断， 返回 true或者false。

```

Predicate<Integer> integerPredicate = x -> {
    if (x > 23) {
        return false;
    }
    return true;
};
integerPredicate.test(t: 25);

```

- **lambda表达式**

- Collection/Map + Lambda


```
List<String> list = Arrays.asList("Tom", "Mary", "Huahua");
```

How to print the values of this list?

```
for(int i = 0; i<list.size(); i++){  
    System.out.println(list.get(i));  
}
```

```
for(String name: list){  
    System.out.println(name);  
}
```

```
list.forEach(name -> System.out.println(name));
```

```
List<StudentInfo> studentList = new ArrayList<>();
```

How to filter data?

```
System.out.println(x: "=====Without FP=====");  
List<StudentInfo> tallBoys = new ArrayList<>();  
for (StudentInfo stu : studentList) {  
    if(stu.getGender()=="m" && stu.getHeight()>=1.7)  
        tallBoys.add(stu);  
}  
for(StudentInfo boy: tallBoys)  
    System.out.println(boy.toString());  
  
System.out.println(x: "=====Using FP=====");  
List<StudentInfo> boys = studentList.stream().filter(s -> s.getGender()  
=="m" && s.getHeight() >= 1.7).collect(Collectors.toList());  
boys.forEach(b->System.out.println(b));
```

```
Map<String, String> maps = new HashMap<String, String>();  
maps.put(key: "a", value: "A");  
maps.put(key: "b", value: "B");  
maps.put(key: "c", value: "C");
```

How to print the key and value of maps?

```
System.out.println(x: "=====Without FP=====");  
for(String key: maps.keySet()){  
    String value = maps.get(key);  
    System.out.println("key: " + key + ", value: " + value);  
}  
  
System.out.println(x: "=====Using FP=====");  
maps.forEach((k, v) -> System.out.println("key: " + k + ", value: " + v));
```

Lecture 11 多线程程序设计

1、概念

程序(program): 一段静态代码, 对数据描述与操作的代码的集合, 应用程序执行的蓝本。

进程(process): 程序的一次执行过程, 是系统运行程序的基本单位。程序是静态的, 进程是动态的。系统运行一个程序即是一个进程从创建、运行到消亡的过程。

多任务(multi task): 在一个系统中可以同时运行多个程序, 即有多个独立运行的任务, 每个任务对应一个进程。

线程: 比进程更小的执行单位, 一个进程中可以包含多个线程。

(1) 一个进程在执行过程中, 为同时完成多项操作, 可产生多个线程, 形成多条执行线索。

(2) 每个线程都有自身产生、存在和消亡的过程。

(3) 一个进程中的所有线程都在该进程的虚拟地址空间中, 使用该进程的全局变量和系统资源。

(4) 操作系统给每个线程分配不同的CPU时间片, 在某一时刻, CPU只执行一个时间片内的线程, 多个线程在CPU内轮流执行。

2、多进程

(1) 多进程环境中每个进程既包括其所要执行的指令, 也包括执行指令所需的系统资源, 如CPU、内存空间、I/O端口等, 不同进程所占用的系统资源相对独立;

(2) 多线程环境中每个线程都隶属于某一进程, 由进程触发执行, 属于同一进程的所有线程共享该进程的系统资源;

(3) 一个进程就是一个应用程序, 有自己的入口和出口; 线程本身没有入口和出口, 其自身也不能独立运行, 它隶属于某个进程, 由进程触发执行, 完成其任务后自动终止, 或由进程使之强制终止。

3、多线程

(1) 多线程程序设计: 单个程序包含并发执行的多个线程。多线程程序执行时, 该程序对应的进程中有多控制流在同时执行, 即具有并发执行的多个线程; 如PV操作, 银行问题, 网络聊天, Web Server接受客户端的请求问题。

(2) 线程之间共享相同的内存单元, 所以线程之间的切换速度和通信远比进程之间快。

(3) 多个线程轮流抢占CPU资源而运行时, 微观上, 一个时间只能有一个作业被执行, 在宏观上可使多个作业被同时执行, 即等同于多台计算机同时工作, 使系统资源特别是CPU的利用率得到提高, 从而可以提高整个程序的执行效率。

4、每个线程都有一个独立的程序计数器和方法调用栈 (method invocation stack)

- 程序计数器：也称为PC寄存器，当线程执行一个方法时，程序计数器指向方法区中下一条要执行的字节码指令。

- 方法调用栈：简称方法栈，用来跟踪线程运行中一系列的方法调用过程，栈中的元素称为栈帧。每当线程调用一个方法，就会向方法栈压入一个新帧，帧用来存储方法的参数、局部变量和运算过程中的临时数据。

5、线程调度：通常是抢占式而不是分时间片式，即线程先抢到CPU资源则先运行。

(1) 一个线程获得执行权后，将持续运行下去，直到运行结束或阻塞，或者有另一个高优先级线程就绪（这种情况称为低优先级线程被高优先级线程所抢占）。

(2) 所有被阻塞的线程按次序排列，组成一个阻塞队列。例如：

-因为需要等待一个较慢的外部设备，例如磁盘或用户。

-让处于运行状态的线程调用Thread.sleep()方法。

-让处于运行状态的线程调用另一个线程的join()方法。

(3) 所有就绪但没有运行的线程，根据其优先级排入一个就绪队列。当CPU空闲时，如果就绪队列不空，就绪队列中第一个具有最高优先级的线程将运行；当一个线程被抢占而停止运行时，它的运行态被改变并放到就绪队列的队尾；一个被阻塞（可能因为睡眠或等待I/O设备）的线程就绪后通常也放到就绪队列的队尾。

(4) 多线程运行时，各个线程通过竞争CPU时间而获得运行机会，各线程得到和占用CPU的时间是不可预测的，一个运行时线程在什么地方被暂停是不确定的。

6、优先级

(1) 线程的调度按其优先级高低顺序执行，同样优先级的线程遵循“先到先执行”原则

(2) 线程创建时，继承父线程的优先级

(3) 线程优先级：范围 1~10（级），数值越大，级别越高

(4) Thread类定义的3个常数

MIN_PRIORITY 最低(小)优先级（值为1）

MAX_PRIORITY 最高(大)优先级（值为10）

NORM_PRIORITY 默认优先级（值为5）

(5) 常用方法

getPriority()：获得线程的优先级

setPriority()：设置线程的优先级

7、主线程

(1) 每当用java命令启动一个Java虚拟机进程 (Application应用程序) , Java虚拟机就会创建一个主线程, 该线程从程序入口main()方法开始执行。

(2) 在主线程中创建Thread类或其子类对象时, 就创建了一个线程对象。Programmer可以控制线程的启动、挂起与终止。

8、线程的状态: (多线程运行时) 新建、就绪runnable、运行running、阻塞blocked、终止

(1) 新建: 一个Thread类或其子类对象被创建, 此时它已经有了相应的内存空间和其他资源。如 Thread myThread = new Thread();

(2) 就绪: 调用start()方法启动处于新建状态的线程后, 将进入线程队列等待CPU服务, 此时它已经具备了运行的条件, 轮到它享用CPU资源时, 就可以脱离创建它的主线程, 开始自己的生命周期。

(3) 运行: 就绪态的线程被调度并获得处理器资源时, 便进入运行状态。每个Thread类及其子类的对象都有一个run()方法, 定义了这个线程的操作和功能。当线程对象被调用执行时, 它将自动调用本对象的run()方法, 从第一句开始顺序执行。

(4) 阻塞: 一个正在执行的线程暂停自己的执行而进入的状态。引起线程由运行状态进入阻塞状态的可能情况及使其返回就绪态的方法:

① -该线程正在等待I/O操作的完成——等待该I/O操作完成

② -调用了该线程的sleep()方法——等待该休眠结束后自动脱离阻塞态

③ -调用了wait()方法——调用notify()或notifyAll()方法

④ -让处于运行状态的线程调用另一个线程的join()方法

(5) 终止:

① -自然终止: 线程完成了自己的全部工作

② -强制终止: 在线程执行完之前, 调用stop()或destroy()方法终止线程

(6) 状态转换:

9、Java中的多线程是建立在Thread类，Runnable接口的基础上的，通常有两种办法让我们来创建一个新的线程：

-创建一个Thread类,或者一个Thread子类的对象

-创建一个实现Runnable接口的类的对象

10、Thread类

(1) 构造方法 (8个)

Thread(); Thread(Runnable target); Thread(Runnable target, String name);

Thread(ThreadGroup group, Runnable target); Thread(ThreadGroup group, String name);

Thread(String name); Thread(ThreadGroup group, Runnable target, String name);

Thread(ThreadGroup group, Runnable target, String name, long stackSize);

说明：name为线程名称；target为要运行的线程对象；group为线程组名称；stackSize为堆栈的大小

(2) 常用方法

方法	说明
currentTread()	返回当前运行的Thread对象
setName()	设置线程名称
getName()	返回线程名称
setPriority()	设置线程优先级
getPriority()	返回线程优先级
start()	启动一个线程
run()	线程体，由start()调用，当run()方法返回时，当前线程结束
stop()	使调用它的线程立即停止执行
isAlive()	如果线程已被启动而未被终止则返回true，若是新创建的或已被终止则返回false
sleep(int n)	使线程睡眠n毫秒，之后再次运行
yield()	用于提示线程调度器当前线程愿意让出CPU资源，以便让其他同优先级或更高优先级的线程获得执行机会
join()	引起线程等待，直至join所调用的线程结束
suspend()	使线程挂起，暂停运行
resume()	恢复挂起的线程，使其处于可运行状态
wait()	使线程等待
notify()	随机唤醒该对象等待池中的一个线程
notifyAll()	唤醒该对象等待池中的所有线程

11、sleep()与yield()

(1) sleep(): 静态实例方法，使线程转入阻塞态，具有更好的可移植性；

(2) yield(): 静态实例方法，使线程转入就绪态，主要用于测试阶段，认为提高程序的并发性。

12、wait()与sleep()

同：都能使线程等待而停止运行

异：sleep()不会释放对象的锁，而wait()方法进入等待时会释放对象的锁，使得其它线程能对这些枷锁的对象进行操作。

13、join()方法：使当前正在运行的线程暂停下来，等待指定的时间后或等待调用该方法的线程结束后，再恢复运行。

(1) 构造方法：

① public final void join() throws InterruptedException;

② public final void join(long millis) throws InterruptedException;

③ public final void join(long millis, int nanos) throws InterruptedException;

(2) 方法join()将引起现行线程等待，直至方法join()所调用的线程结束。

14、创建线程

(1) 用线程类（Thread的子类）创建线程

线程体：public void run()方法，是整个线程的核心，线程所要完成任务的代码都定义在线程体中，不同功能的线程之间的区别就在于它们的线程体不同。

(2) 用Runnable接口创建线程

a. 从根本上讲，任何实现线程功能的类都必须实现Runnable接口

b. Thread(Runnable target); Thread(Runnable target, String name);

c. Runnable接口中只定义了一个方法run()，即方法体。

d. 适用情况：Java只允许单继承，如果一个类已经继承了Thread就不能再继承其他类；在除了run()之外不打算再重写Thread类的其他方法的情况

15、主线程与其他线程共存，可以用join()方法保证主线程最后结束。（见OOP课件多线程程序设计-P54-P59）

16、终止线程

(1) 当线程执行完run()方法，将自动终止运行

(2) 实际编程中，定义一个标志变量，通过程序改变标志变量的值，从而控制线程从run()方法中退出。

17、创建用户多线程的步骤

(1) 方式一

① 创建一个Thread类的子类；

② 在子类中将希望该线程做的工作写到run()里；

③ 生成该子类的一个对象；

④ 调用该对象的start()方法。

(2) 方式二

① 创建一个实现Runnable接口的类；

② 在该类中将希望该线程做的工作写到run()里面；

③ 生成该类的一个对象；

④ 用上述对象去生成Thread类的一个对象；

⑤ 调用Thread类的对象的start()方法；

(3) 方式三

① 创建一个实现Runnable接口的类；

② 在该类中将希望该线程做的工作写到run()里；

③ 写一个start()方法，在里面创建Thread，调用start()；

④ 生成Runnable类的一个对象；

⑤ 调用该对象的start()方法。

18、同步synchronized

(1) 同步：一种保证共享资源完整性的手段，在代表原子操作的程序代码前加上synchronized标记，这样的代码成为同步代码块。

(2) synchronized关键字可以用在成员方法上、静态方法上、语句块上。

(3) synchronized的用法：

① public synchronized void write();

- 由synchronized关键字锁定的是一个对象,而不是一个方法。

- 一个线程调用某个对象的synchronized方法,就锁定了这个对象，直到它从这个方法中退出,才释放了这个锁定。因此一旦一个对象被某个线程锁定了,它的所有synchronized方法就都不允许别的线程调用了。

- 如果一个线程调用该方法，则方法体中访问的对象都被锁定，不允许其他线程访问，以解决共享资源的访问冲突问题。

② public static synchronized int getValue();

一个线程调用一个类的static synchronized方法,就锁定了这个类对象,也就控制了所有static synchronized方法。即类中所有的静态同步方法在每个时刻只有一个可以执行。

③ Synchronized(obj){...}

一个线程进入由synchronized修饰的程序块,就锁定了由obj指定的那个对象。

(4) 一个对象在某一时刻只能被一个线程锁定; 一个类可以有多个对象, 不同的对象可以被不同的线程锁定; 一个线程可以锁定多个对象。

(5) synchronized方法或synchronized区段内的代码在同一时刻下可有多个线程执行, 只要是对不同的对象调用该方法就可以。

19、线程通信: 线程之间交换信息, 简单通信可用wait()、notify()方法

(1) wait(): 执行该方法的线程释放对象的锁, Java虚拟机将其放到该对象的等待池中等待其他线程将它唤醒。

(2) notify(): 执行该方法的线程随机唤醒在对象的等待池中等待的一个线程, Java虚拟机从对象的等待池中随机选择一个线程, 把它转到对象的锁池中。

20、一般每个共享对象的互斥锁存在两个队列: 锁等待队列和锁申请队列。锁申请队列中的第一个线程可以对该共享对象进行操作, 锁等待队列中的线程在某些情况下将移入锁申请队列中。

21、wait(), notify(), notifyAll()都是与同步相关联的方法, 必须在已持有锁的情况下执行, 即只能出现在synchronized修饰的方法中, 不同步的方法或代码中则使用sleep()暂停线程。一个“在同步方法中等待”的线程自己不能恢复运行, 只能等另一个也进入了该对象的synchronized方法的线程调用notify和notifyAll后才可能恢复运行。

wait(): 释放已持有的锁, 进入等待队列

notify(): 唤醒wait队列中的第一个线程并把它移入锁申请队列

notifyAll(): 唤醒wait队列中的所有线程并全部移入锁申请队列

22、线程间要传送的数据较多时, 必须使用共享主存、管道流等通信方式

23、管道流通信

(1) 管道: 用来把一个程序的输出连接到另一个程序的输入, java.io中提供了类

(2) 管道的输入/输出部件: PipedInputStream / PipedOutputStream

(3) 构造方法中, 管道的输入/输出流的参数为对应管道的输出/输入流

(4) 管道输入/输出流还可以用connect()方法进行连接

24、死锁: 由于程序设计不合理而造成程序中所有的线程都陷入等待态或阻塞态, 发生于多线程竞争使用多资源的程序中, 几个线程循环等待另一个线程所持有的锁。

(1) 死锁不是由于资源短缺造成的

(2) 每个线程都不能继续运行, 除非另一线程运行完同步程序块。

(3) 因为哪个线程都不能继续运行，所以哪个线程都无法运行完同步程序块。

(4) 另一种情况是在wait、notify结构中，如果所有的线程都wait了，就不会有线程来notify

25、后台线程：又称守护线程，为其它线程提供服务的线程。

(1) 守护线程不是应用程序的核心部分，当应用程序的所有非守护线程终止运行时，即是仍然有守护线程在运行，应用程序也将终止。只要有非守护线程在运行，应用程序就不会终止。

(2) 主线程和由前台线程创建的线程默认为前台线程。

(3) 通过调用isDaemon()方法来判断一个线程是否是守护线程

(4) 通过调用setDaemon()方法将一个线程设为守护线程

26、定时器Timer

(1) TimerTask类表示定时器执行的一项任务

(2) Timer类的schedule(TimerTask task, long delay, long period)方法用于设置定时器需要定时执行的任务。参数task表示任务；delay表示延迟执行的时间(ms)，即多久后开始执行第一次；period表示每次执行任务的间隔时间(ms)。

27、多线程程序对系统的影响

(1) 线程需要占用内存

(2) 线程过多，会消耗大量CPU时间来跟踪线程

(3) 多线程同时访问共享资源的问题，协调不好会发生死锁和资源竞争

(4) 同一个任务的所有线程都共享相同的地址空间和任务的全局变量，需要考虑

Lecture 12 网络编程
